// Security Assessment

02.18.2026 - 03.24.2026

Rayls Network Node

RAYLSFUSION LTD. (ParFin)

HALBORN

Rayls Network Node - RAYLSFUSION LTD. (ParFin)

Prepared by: HALBORN

Last Updated 04/20/2026

Date of Engagement: February 18th, 2026 - March 24th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
22	1	2	7	6	6

TABLE OF CONTENTS

1. Introduction

`ParFin` engaged Halborn to conduct a security assessment of the **Rayls Network** node implementation, beginning on February 18th, 2026, and concluding on March 25th, 2026. This security assessment was scoped to the Rust source code of the Github repository, focusing on the node's networking, worker, state synchronization, execution, orchestrator, storage, and shared types modules. Commit hashes and further details can be found in the Scope section of this report.

The **Rayls Network** node is responsible for receiving and propagating transactions, building and attesting batches, ordering consensus outputs through the primary/worker architecture, synchronizing missing consensus and epoch data, executing ordered outputs deterministically, and coordinating epoch transitions across the execution and consensus layers. It must enforce strict consistency between networking, consensus, execution, and persistence while remaining resilient to adversarial peers, malformed inputs, restart conditions, and resource-exhaustion scenarios.

2. Assessment Summary

The team at Halborn assigned a full-time security engineer to verify the security of the project. The security engineer is a blockchain and Rust security expert with advanced knowledge of consensus protocols, distributed systems, peer-to-peer networking, state machine execution, and denial-of-service resilience.

The purpose of this assessment is to:

- Ensure that the node correctly enforces protocol invariants across the modules.
- Identify potential vulnerabilities related to peer messaging, epoch transitions, recovery logic, persistence guarantees, and inter-module coordination.
- Verify the correctness and security of batch propagation, header voting, certificate handling, consensus output processing, and execution handoff.
- Confirm that storage integrity, committee transitions, timeout behavior, and concurrency boundaries are properly enforced to prevent safety, liveness, or resource-exhaustion failures.

In summary, Halborn identified several recommendations to strengthen the node's resilience and operational safety, which were mostly addressed by `ParFin` team. The most relevant ones are the following:

- `Add explicit revocation of the trusted flag for rotated-out validators during epoch transitions to prevent permanent is_trusted=true state and restore correct penalty, scoring, and connection handling behavior.`
- `Validate and clamp externally supplied Kademlia NodeRecord timestamps; enforce bounded clock skew, use local insertion time for eviction ordering, and rate-limit PUT requests to prevent peer eviction and record freezing.`
- `Apply penalties to the original message author (message.source) instead of relay peers (propagation_source), ensuring correct attribution across all gossip validation and processing layers.`
- `Enforce per-sender deduplication in epoch vote alternative record counters to prevent a single validator from inflating counts and forcing the system into the recovery path.`
- `Validate externally received epoch records against locally trusted state (e.g., parent hash and expected committee) before acceptance, removing reliance on self-referential verification during recovery.`
- `Validate transaction payload lengths before slice access and handle malformed TXN gossip safely to prevent remote-triggered panics and ensure proper error propagation and penalization.`
- `Correct IP-based ban removal logic during epoch transitions to avoid lifting bans for unrelated peers sharing the same IP and maintain consistency of ban counters.`
- `Verify BLS signatures of all peer-fetched consensus data before caching or promoting it into storage to prevent poisoning of local state.`

- `Ensure consistent batch deduplication across all execution paths, including parked batch drain flows, to prevent divergent execution and state inconsistencies between nodes.`

3. Test Approach And Methodology

Halborn performed a combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation, automated testing techniques help enhance code coverage and quickly identify items that do not follow security best practices. The following phases and associated tools were used during the assessment:

- Research into the architecture and security assumptions of the Rayls node implementation.
- Manual code review and walkthrough of the **network**, **worker**, **orchestrator**, **execution**, **processor**, **bridge**, **state-sync**, **storage**, and **types** modules.
- Assessment of critical Rust control flows related to epoch transition, restart recovery, certificate handling, and consensus/execution coordination.
- Review of trust boundaries between peer-provided data and locally derived execution or committee state.
- Validation of inter-module communication, including Worker ↔ Primary, Consensus ↔ Bridge/Processor, State Sync ↔ Storage, and Orchestrator ↔ Execution.
- Targeted analysis of denial-of-service, crash-consistency, and liveness risks in asynchronous tasks, bounded queues, retry loops, and recovery paths.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

FILE

(a) Submitted File: axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11.zip

(b) Items in scope:

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/all_peers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/banned.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/behavior.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/cache.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/manager.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/peer.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/score.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/status.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/types.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/banned_peers.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/cache_peers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/codec_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/common.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/network_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/peer_manager.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/peers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/types.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/codec.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/kad.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/types.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/src/epoch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/src/lib.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/batch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/test_utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/tests/it/build_batches.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/src/validator.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/handler.rs

- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/message.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/mod.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/batch_provider_tests.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/main.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/network_tests.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/quorum_waiter_tests.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/batch_fetcher.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch_fetcher.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/lib.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/metrics.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/quorum_waiter.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/test_utils.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/src/worker.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/consensus/worker/Cargo.toml
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/execution/evm/benches/lib.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/block.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/config.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/context.rs
- /axyl-068cb4a881621b9df5700ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/factory.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/handler.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/abi.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/composite_inspector.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/inspector.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/precompile.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/storage.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/bypass_validator.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/chainspec.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/dirs.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/payload.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/persistence.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/rpc_server_args.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/system_calls.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/test_utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/traits.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/txn_pool.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/worker.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mdbx/database.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mdbx/metrics.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mdbx/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/database.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/metrics.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/wraps.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/certificate_store.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/checkpoint_store.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/consensus_store.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/epoch_store.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/payload_store.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/proposer_store.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/vo

te_digest_store.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/layered_db.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mem_db.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/src/errors.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/src/subscriber.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/engine/builder.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/engine/inner.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/engine/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/tests/epoch_transition_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/epoch_transition.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/health.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/manag

er.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/primary.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/worker.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/src/payload_builder.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/bls_keypair.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/bls_public_key.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/bls_signature.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/intent.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/net

work.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/block.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/certificate.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/epoch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/header.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/info.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/output.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/reputation.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/vote.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/test_utils/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/test_utils/tracing.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/worker/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/worker/pending_batch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/worker/sealed_batch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/batch_tracker.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/codec.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/committee.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/database_traits.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/gas_accumulator.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/genesis.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/helpers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/notifier.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/serde.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/sync.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/task_manager.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/Cargo.toml

REMEDIATION COMMIT ID:

- https://github.com/raylsnetwork/axyl/pull/280
- 3289871
- 637fbf6
- https://github.com/raylsnetwork/axyl/pull/292
- https://github.com/raylsnetwork/axyl/pull/266
- https://github.com/raylsnetwork/axyl/pull/283
- 00a3ced
- b77e45c
- https://github.com/raylsnetwork/axyl/pull/273
- https://github.com/raylsnetwork/axyl/pull/226
- 66ff1aa

- 4ecbe36
- 892a2ba
- 154124c
- https://github.com/raylsnetwork/axyl/pull/238
- dbbef41
- https://github.com/raylsnetwork/axyl/pull/275
- e3ed35e
- https://github.com/raylsnetwork/axyl/pull/270
- https://github.com/raylsnetwork/axyl/pull/310

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
1	2	7	6

INFORMATIONAL
6

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
EXTERNALLY-TRIGGERED KADEMLIA STORE EXHAUSTION PROPAGATES FATAL ERROR TO CRITICAL NETWORK TASK CAUSING NODE SHUTDOWN	CRITICAL	SOLVED - 03/29/2026
STALE IS_TRUSTED FLAG GRANTS PERMANENT IMMUNITY TO ROTATED-OUT VALIDATORS	HIGH	SOLVED - 03/05/2026
UNVALIDATED SELF-REPORTED NODERECORD TIMESTAMPS ENABLE COMMITTEE PEER EVICTION	HIGH	SOLVED - 03/17/2026
GOSSIP REJECTION PENALIZES HONEST RELAY PEER INSTEAD OF MALICIOUS AUTHOR	MEDIUM	SOLVED - 04/02/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MISSING PER-SENDER DEDUPLICATION IN EPOCH VOTE ALT-RECS COUNTER ENABLES SINGLE VALIDATOR TO FORCE RECOVERY PATH	MEDIUM	SOLVED - 03/24/2026
SELF-REFERENTIAL EPOCH RECORD VALIDATION ENABLES FAKE EPOCHRECORD INJECTION VIA RECOVERY PATH	MEDIUM	SOLVED - 04/01/2026
UNGUARDED SLICE IN FUNCTION TRIGGERS REMOTE PANIC VIA MALFORMED TXN GOSSIP	MEDIUM	SOLVED - 03/05/2026
EPOCH TRANSITION CLEARS NON-VALIDATOR IP BANS AND PERMANENTLY DESYNCHRONIZES BAN COUNTER	MEDIUM	SOLVED - 04/03/2026
UNVERIFIED BLS SIGNATURES ON PEER-FETCHED CONSENSUS ENABLE BYZANTINE CACHE POISONING	MEDIUM	SOLVED - 03/26/2026
PARKED BATCH DRAIN PATH BYPASSES DEDUPLICATION GUARD ENABLING STATE DIVERGENCE	MEDIUM	SOLVED - 03/17/2026
EPOCH COMMITTEE SELECTION SEED DERIVED UNILATERALLY FROM BOUNDARY LEADER	LOW	RISK ACCEPTED - 04/03/2026
THE CRITICAL HEALTHCHECK TASK MAY TRIGGER A FULL NODE SHUTDOWN	LOW	SOLVED - 03/05/2026
QUORUMWAITER REMAINING-TIME SUBTRACTION INVERSION PREVENTS PEER DRAIN TASK FROM SPAWNING	LOW	SOLVED - 03/17/2026
INFINITE RETRY LOOP IN BATCHFETCHER::FETCH ENABLES LIVENESS DENIAL-OF-SERVICE VIA BATCH WITHHOLDING	LOW	SOLVED - 03/24/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
EIP-1559 FEE MARKET PERMANENTLY DISABLED	LOW	SOLVED - 03/17/2026
VALIDATOR SLASHING IS NOT IMPLEMENTED IN THE EXECUTION LAYER	LOW	RISK ACCEPTED - 04/03/2026
INTEGER TRUNCATION IN EPOCH COMMITTEE VALIDATION WEAKENS THE MAJORITY THRESHOLD	INFORMATIONAL	SOLVED - 03/22/2026
REDUNDANT GOSSIP TOPIC SUBSCRIPTION	INFORMATIONAL	SOLVED - 03/05/2026
WORKER GOSSIP HANDLER REDUNDANTLY PROCESSES INVALID MESSAGES TWICE	INFORMATIONAL	SOLVED - 03/26/2026
MAINNET GENESIS MISSING HARDFORK TIMESTAMPS	INFORMATIONAL	SOLVED - 04/02/2026
BATCH DIGEST GOSSIPED BEFORE PERMANENT STORAGE COMMIT	INFORMATIONAL	SOLVED - 03/26/2026
UNCHECKED INTEGER ARITHMETIC ACROSS MULTIPLE MODULES	INFORMATIONAL	SOLVED - 04/02/2026

7. FINDINGS & TECH DETAILS

7.1 EXTERNALLY-TRIGGERED KADEMLIA STORE EXHAUSTION PROPAGATES FATAL ERROR TO CRITICAL NETWORK TASK CAUSING NODE SHUTDOWN

// CRITICAL

Description

The Kademlia store implementation (`KadStore`) enforces two hard capacity limits (`max_records` and `max_provided_keys`) returning `Err(Error::MaxRecords)` or `Err(Error::MaxProvidedKeys)` respectively when these thresholds are exceeded. Both error variants are propagated unmodified via the `?` operator from the store layer through the Kademlia event handler and into the critical network task loop, where their occurrence is interpreted by the `TaskManager` as an abnormal termination of a critical task, triggering the node's orderly shutdown procedure.

Two distinct externally-reachable code paths can intentionally bring these conditions about:

- **Path A — `AddProvider` (no authentication required):** The `InboundRequest::AddProvider` branch in `process_kad_event` stores provider records with no validation of any kind: no authentication of the advertising peer, no BLS signature verification, no rate limiting, and no content filtering. Any peer that has completed a QUIC handshake can send approximately 1024 `ADD_PROVIDER` protocol messages, exhaust `max_provided_keys`, and cause `add_provider()` to return `Err(MaxProvidedKeys)`. This error propagates via `?` to the network loop, which terminates the critical task and initiates node shutdown.
- **Path B — `PutRecord` (BLS signature required):** The `InboundRequest::PutRecord` handler validates BLS signatures and publisher identity before invoking `store_mut().put()`. However, BLS keypair generation is computationally trivial and requires no external registration or permissioning. An attacker can generate 1024 distinct BLS keypairs, produce a valid signed `NodeRecord` per keypair, and submit sequential `PUT_VALUE` Kademlia messages. Upon reaching the store capacity, any subsequent valid `PUT` causes `put()` to return `Err(MaxRecords)`, which is mapped to `NetworkError::StoreKademliaRecord` and propagated via `?` into the network loop. Since valid records incur no penalty, the attacker accumulates entries undetected.

Both the primary and worker consensus networks operate independent `KadStore` instances with identical default limits, making each individually vulnerable.
The underlying design tension is that these store limits exist precisely to bound memory and I/O growth, they are a legitimate defensive measure. The problem is that their error handling strategy is symmetric with that of genuine unexpected internal failures: rather than treating store exhaustion as an expected operational condition that should be handled gracefully at the application boundary (with peer penalization and record rejection), the error is allowed to propagate upward and activate the same shutdown path that would be triggered by an unrecoverable internal fault.

Code Location

Code of `process_kad_event` function from `crates/consensus/network/src/consensus.rs` file:

```
fn process_kad_event(&mut self, event: kad::Event) -> NetworkResult<()> {
    match event {
        kad::Event::InboundRequest { request } => {
            trace!(target: "network-kad", "inbound {request:?}");
            match request {
                kad::InboundRequest::FindNode { num_closer_peers: _ } => {}
                kad::InboundRequest::GetProvider { .. } => {}
                kad::InboundRequest::AddProvider { record } => {
                    if let Some(record) = record {
                        self.swarm
                            .behaviour_mut()
                            .kademlia
                            .store_mut()
                            .add_provider(record)
                            .map_err(|e| NetworkError::StoreKademliaRecord(e.to_string()))?; /
                    }
                }
                kad::InboundRequest::GetRecord { .. } => {}
                kad::InboundRequest::PutRecord { source, connection: _, record } => {
                    if let Some(record) = record {
                        self.process_kad_put_request(source, record)?; // ← MaxRecords trigger
                    }
                }
            }
        }
        // ...
    }
    Ok(())
}
```

Code of `process_kad_put_request` function from `crates/consensus/network/src/consensus.rs` file:

```
fn process_kad_put_request(
    &mut self,
    source: PeerId,
    record: kad::Record,
) -> NetworkResult<()> {
    // ban checks and BLS signature validation...
    if let Some((key, value)) = self.peer_record_valid(&record) {
        if self.is_newer_record(&record) {
            self.swarm
                .behaviour_mut()
                .kademlia
                .store_mut()
                .put(record)
                .map_err(|e| NetworkError::StoreKademliaRecord(e.to_string()))?; // ← MaxRecor
            // ...
        }
    }
    Ok(())
}
```

Code of `add_provider` function from `crates/consensus/network/src/kad.rs` file:

```
fn add_provider(&mut self, record: ProviderRecord) -> libp2p::kad::store::Result<()> {
    if self.config.max_provided_keys == self.num_providers {
        return Err(Error::MaxProvidedKeys); // ← global limit reached
    }
    // ...
    if !found {
        if recs.len() >= self.config.max_providers_per_key {
```

```
316                 return Err(Error::MaxProvidedKeys); // ← per-key limit reached
317             }
318             recs.push(kr);
319         }
320         // ...
321         match self.kad_type {
322             KadStoreType::Primary => self.db
323                 .insert::<KadProviderRecords>(&key, &encode(&records))
324                 .map_err(|_| libp2p::kad::store::Error::ValueTooLarge)?,
325             KadStoreType::Worker => self.db
326                 .insert::<KadWorkerProviderRecords>(&key, &encode(&records))
327                 .map_err(|_| libp2p::kad::store::Error::ValueTooLarge)?,
328         }
329         Ok(())
330 }
```

Impact

A remote peer with a single active QUIC connection (requiring no committee membership, no stake, and no prior trust relationship) can exhaust the Kademlia provider store (`max_provided_keys = 1024`) by transmitting approximately 1024 unauthenticated `ADD_PROVIDER` protocol messages over the established connection.

Upon exhaustion, `add_provider()` returns `Err(MaxProvidedKeys)`, which propagates via the `?` operator through `process_kad_event` into `ConsensusNetwork::run()`. The `run()` function returns this error to the `spawn_critical_task` closure in `EpochManager::spawn_node_networks()`. The `TaskManager` then detects that a critical task has exited (regardless of whether the exit was the result of an external condition or an internal fault) and proceeds through its standard shutdown sequence: `join_internal()` raises `TaskJoinError::CriticalExitOk`, `join_until_exit()` returns `Err`, `EpochManager::run()` wraps it in an `eyre!` error, `launch_node()` surfaces it to `main()`, and `main()` calls `std::process::exit(1)`.

The node performs an orderly but complete shutdown. An attacker who repeats this sequence after node restart achieves a persistent denial of service. Both the primary and worker consensus networks are independently vulnerable; exhausting the store on either is sufficient to initiate shutdown.

Proof of Concept

Scenario

The test simulates the condition that an external adversary would produce after exhausting the Kademlia provider store.
The `KadStore` is pre-filled with 1024 `ProviderRecord` entries (the default value of `max_provided_keys)` replicating the state the store reaches after a connected peer sends 1024 unauthenticated `ADD_PROVIDER` protocol messages. A subsequent inbound `AddProvider` event is then constructed and delivered directly to `process_kad_event`, replicating the code path that the network event loop executes on every swarm event. The test verifies that this externally-reachable condition causes `process_kad_event` to return `Err(NetworkError::StoreKademliaRecord)`, the exact error value that the `?` operator in `ConsensusNetwork::run()` would propagate to the critical task, initiating the node's shutdown sequence.

Test Code

Copy Code

```
#[tokio::test]
async fn poc_kad_overflow_critical_task_terminates_with_error() {
    use libp2p::kad::{self, ProviderRecord, RecordKey};
    use tokio::sync::mpsc as tokio_mpsc;

    let TestTypes { peer1, .. } = create_test_types::<TestPrimaryRequest, TestPrimaryResponse>();
    let mut network = peer1.network;

    // Pre-fill KadStore to the limit before handing network to the task.
    {
        let store = network.swarm.behaviour_mut().kademlia.store_mut();
        for _ in 0..1024 {
            let record = ProviderRecord {
                key: RecordKey::new(&PeerId::random().to_bytes()),
                provider: PeerId::random(),
                expires: None,
                addresses: vec![],
            };
            store.add_provider(record).expect("fill ok");
        }
    }

    // Directly call process_kad_event with the overflow event.
    // This replicates what run()'s event loop calls on the swarm's next event.
    let overflow_event = kad::Event::InboundRequest {
        request: kad::InboundRequest::AddProvider {
            record: Some(ProviderRecord {
                key: RecordKey::new(&PeerId::random().to_bytes()),
                provider: PeerId::random(),
                expires: None,
                addresses: vec![],
            }),
        },
    };

    let result = network.process_kad_event(overflow_event);

    // This is what run() would receive from `?` and return to the task:
    assert!(
        matches!(result, Err(NetworkError::StoreKademliaRecord(_))),
        "process_kad_event must return Err(StoreKademliaRecord) at capacity"
    );

    // Confirm: in run(), `self.process_event(event).await?` would return this
    // error, causing run() to return Err(NetworkError::StoreKademliaRecord).

    let err_msg = result.unwrap_err().to_string();
    assert!(
        err_msg.contains("kad record"),
        "Error message surfaced to operator logs: '{err_msg}'"
    );
}
```

Result

The test confirms that `process_kad_event` returns `Err(NetworkError::StoreKademliaRecord)` when the provider store is at capacity and an additional `AddProvider` request is received. This is the precise error type that, in production, propagates via `?` through `ConsensusNetwork::run()` → `spawn_critical_task` → `TaskManager::join_internal`, where the node's shutdown procedure is initiated.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (10.0)

Recommendation

It is recommended to decouple the handling of Kademlia store capacity limits from the node's fatal error propagation path. Specifically:
1. Replace the `?` operator on `add_provider()` and `store_mut().put()` with explicit `match` arms that treat `MaxRecords` and `MaxProvidedKeys` as expected operational boundaries: log the condition, apply penalty to the source peer, and return `Ok(())` to allow the network loop to continue processing.
2. Extend the same validation logic currently applied to `PutRecord` to the `AddProvider` handler, which currently accepts provider records unconditionally from any connected peer.
3. Introduce per-peer inbound rate limiting for Kademlia protocol messages to prevent any single peer from driving the store toward exhaustion within a short time window.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by removing the propagation error in the max store capacity parameters.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/280

7.2 STALE IS_TRUSTED FLAG GRANTS PERMANENT IMMUNITY TO ROTATED-OUT VALIDATORS

// HIGH

Description

The **peer reputation system** assigns a trusted flag (`is_trusted`) and a maximum score (`Score::new_max()`) to every committee member at epoch start. When an epoch transition occurs, `new_epoch()` correctly clears and repopulates `current_committee` and `current_committee_keys` from the incoming committee list. However, the function only iterates over the new committee members to call `make_trusted()` on them; it never iterates over the existing peers map to revoke the trusted flag from peers that were in the previous committee but are absent from the new one.

As a consequence, a former committee member that has been rotated out retains `is_trusted = true` indefinitely inside its Peer struct in every node that connected to it during the prior epoch. This stale flag propagates through four independent security mechanisms, each of which reads `is_trusted` or `peer_is_important()` independently:

1. **Penalty immunity:** `Peer::apply_penalty()` contains an early-exit guard on `is_trusted`. Because the former member still has `is_trusted = true`, every call to `apply_penalty` (including Penalty::Fatal) is a complete no-op. The score never decreases, and `AllPeers::process_penalty()` returns `PeerAction::NoAction` at every invocation because new_reputation == prior_reputation. No disconnect or ban is ever triggered.
2. **Score decay bypass:** The heartbeat timer skips score decay for trusted peers. The former member's score therefore remains pinned at its maximum indefinitely, meaning no reputation-driven disconnection or ban is ever initiated by the background maintenance routine.
3. **Connection slot occupation:** `prune_connected_peers()` explicitly excludes peers where `is_peer_validator() || is_trusted()` is true from the eviction candidate list. With `is_trusted = true`, the rotated-out peer permanently occupies one of the ~39 available connection slots and can never be evicted by the pruning mechanism, regardless of its behavior or the pressure from new legitimate peers.
4. **Gossip authorization bypass:** `verify_gossip()` contains two independent fallback acceptance paths that call `peer_is_important()`. That function returns true when `is_peer_validator(peer_id) || peer.is_trusted()`. Since `is_peer_validator()` checks membership in the live current committee set (correctly updated after rotation), this returns false for the former member. But because `is_trusted` is stale, `peer_is_important()` still resolves to true, and both fallback paths accept the gossip message. This happens even though the former member's BLS key is not present in `authorized_publishers` for the current epoch.

The consequence is that a Byzantine actor who was a legitimate committee member in any prior epoch can, after being rotated out, continue to inject gossip on any consensus topic on nodes that knew it during the active epoch, accumulate zero penalties, and maintain a permanent connection slot indefinitely without any cryptographic capability beyond its original committee identity.

Code Location

Code of the `new_epoch` function from crates/consensus/network/src/peers/all_peers.rs file:

```
pub(super) fn new_epoch(
    &mut self,
    committee: Vec<(BlsPublicKey, NetworkInfo)>,
) -> Vec<(PeerId, PeerAction)> {
    // update current committee
    self.current_committee.clear();          // [!] live set cleared correctly
    self.current_committee_keys.clear();     // [!] live key map cleared correctly
                                             //     but is_trusted in Peer structs of rotated-o
                                             //     validators is NEVER reset — no loop over pe

    let mut actions = Vec::with_capacity(committee.len());
    for (bls_key, NetworkInfo { pubkey, multiaddrs: addr, .. }) in committee {
        let peer_id: PeerId = pubkey.clone().into();
        self.current_committee.insert(peer_id);
        self.current_committee_keys.insert(bls_key, Some(peer_id));

        // ...
        // ...
        // ...

        if let Some(peer) = self.peers.get_mut(&peer_id) {
            peer.make_trusted();             // [!] only incoming members are made trusted;
                                             //     outgoing members are never made untrusted
            peer.update_listening_addrs(addr);
            self.banned_peers.remove_validator_ip(&peer_id, peer.known_ip_addresses());
        }
    }
    actions
}
```

Code of the `apply_penalty` function from crates/consensus/network/src/peers/peer.rs file:

```
pub(super) fn apply_penalty(&mut self, penalty: Penalty) -> Reputation {
    if !self.is_trusted {   // [!] stale is_trusted = true makes every Penalty::Fatal a no-op
        self.score.apply_penalty(penalty);
    }
    self.reputation()
}
```

Code of the `peer_is_important` function from crates/consensus/network/src/peers/manager.rs file:

```
pub(crate) fn peer_is_important(&self, peer_id: &PeerId) -> bool {
    self.is_peer_validator(peer_id)          // false after rotation (checks current_committee)
        || self.peers.get_peer(peer_id).map(|p| p.is_trusted()).unwrap_or_default()
        // [!] stale is_trusted = true causes this to resolve to true post-rotation
}
```

Code of the `verify_gossip` fallback paths from crates/consensus/network/src/consensus.rs file:

```
let auth_config = self.authorized_publishers.get(topic.as_str());

// Path A — startup grace period
if auth_config.is_none() {
    if let Some(source_id) = gossip.source {
        if self.swarm.behaviour().peer_manager.peer_is_important(&source_id) {
            // [!] stale-trusted peer accepted even with no authorization config
            return GossipAcceptance::Accept;
        }
```

```
    }
}

// ... BLS key authorization check ...

} else {
    // Path B — BLS mapping unavailable fallback
    if let Some(source_id) = gossip.source {
        if self.swarm.behaviour().peer_manager.peer_is_important(&source_id) {
            // [!] stale-trusted peer accepted even though BLS key not in authorized_publisher
            return GossipAcceptance::Accept;
        }
    }
    GossipAcceptance::Reject
}
```

Code of `prune_connected_peers` from crates/consensus/network/src/peers/manager.rs file:

Copy Code

```
fn prune_connected_peers(&mut self) {
    let connected_peers = self.peers.connected_peers_by_score_and_routability();
    let mut excess_peer_count =
        connected_peers.len().saturating_sub(self.config.target_num_peers);
    // ...
    let ready_to_prune = connected_peers
        .iter()
        .filter_map(|(peer_id, peer)| {
            if !self.is_peer_validator(peer_id) && !peer.is_trusted() {
                // [!] stale is_trusted = true excludes former validator from eviction indefin
                Some(**peer_id)
            } else {
                None
            }
        })
        .collect::<Vec<_>>();
    // ...
}
```

Impact

A Byzantine actor that was a legitimate committee member in any prior epoch retains, after rotation, a set of compounding immunities on every node it was connected to during its active epoch:

- **Permanent connection presence:** The rotated-out peer is never evicted by the pruning mechanism, permanently consuming one of the ~39 available inbound connection slots. In a network with a small committee and a fixed connection limit, this starves legitimate new peers from connecting.
- **Complete penalty immunity:** No amount of misbehavior (submitting invalid certificates, sending malformed messages, triggering protocol violations) results in a score change, disconnect, or ban on nodes where the stale flag persists. The attacker is effectively beyond reach of the reputation system.
- **Unauthorized gossip injection:** The stale flag causes both fallback paths in verify_gossip() to accept the attacker's gossip on any consensus topic, even though its BLS key is no longer in the epoch's authorized_publishers. Injected messages are forwarded to the application layer and to gossipsub mesh neighbors as if they came from a legitimate current validator.
- **Compounding across epochs:** Because the flag is never revoked, the attack surface grows with each epoch rotation that includes the attacker, and shrinks only when an affected node restarts and loses in-memory peer state.

Proof of Concept

Scenario

This PoC demonstrates that a validator who was part of the committee in epoch N and is rotated out in epoch N+1 retains the `is_trusted = true` flag and maximum reputation score after the epoch transition.
The test simulates the following sequence:
1. A peer **B** is added as a trusted committee member in epoch N.
2. An epoch transition occurs to epoch N+1, where **B is no longer part of the committee**.
3. The system verifies:

- B is no longer recognized as a validator.
- B still retains `is_trusted = true`.
- B retains `Score::new_max()` (maximum reputation score).

4. A `Penalty::Fatal` is applied to B to simulate protocol misbehavior.
5. The test confirms that the penalty has no effect, demonstrating penalty immunity.
This directly proves that the trusted state is not revoked during epoch rotation and that a rotated-out validator maintains privileged network status indefinitely.

Test Code

Copy Code

```
#[test]
fn test_poc_stale_trust_persists_for_rotated_out_committee_member() {
    use rayls_infrastructure_types::now;

    let config = ScoreConfig::default();

    let mut all_peers = create_all_peers(None);

    // B: Byzantine node — committee member in epoch N, rotated out in epoch N+1.
    let mut rng_b = StdRng::from_seed([0xBBu8; 32]);
    let bls_b = *BlsKeypair::generate(&mut rng_b).public();
    let net_b: NetworkPublicKey = NetworkKeypair::generate_ed25519().public().into();
    let peer_id_b: PeerId = net_b.clone().into();
    let addr_b = create_multiaddr(None);

    // C: Honest committee member in epoch N+1 (B is NOT included).
    let mut rng_c = StdRng::from_seed([0xCCu8; 32]);
    let bls_c = *BlsKeypair::generate(&mut rng_c).public();
    let net_c: NetworkPublicKey = NetworkKeypair::generate_ed25519().public().into();
    let addr_c = create_multiaddr(None);

    // Epoch N: mark B trusted (mirrors committee trust assignment).
    all_peers.add_trusted_peer(bls_b, net_b, vec![addr_b.clone()]);
    all_peers.update_connection_status(
        &peer_id_b,
        NewConnectionStatus::Connected {
            multiaddr: addr_b,
            direction: ConnectionDirection::Incoming,
        },
    );

    // Rotate to epoch N+1 committee (excluding B).
    all_peers.new_epoch(vec![(
        bls_c,
        NetworkInfo { pubkey: net_c, multiaddrs: vec![addr_c], timestamp: now() },
```

```
    )]);

    // B is no longer a validator, but remains trusted (stale trust).
    assert!(
        !all_peers.is_peer_validator(&peer_id_b),
        "B must not be a validator after rotation"
    );
    let b_after_rotation = all_peers.get_peer(&peer_id_b).unwrap();
    assert!(
        b_after_rotation.is_trusted(),
        "Vulnerability: B retains stale is_trusted=true after epoch rotation"
    );
    assert_eq!(
        b_after_rotation.score().aggregate_score(),
        config.max_score,
        "B retains max score after epoch rotation"
    );

    // Penalty immunity: even if the relay tried to penalize B later, it is a no-op.
    let penalty_action = all_peers.process_penalty(&peer_id_b, Penalty::Fatal);
    assert!(
        matches!(penalty_action, PeerAction::NoAction),
        "Fatal penalty against stale-trusted B is a no-op"
    );
    assert_eq!(
        all_peers.get_peer(&peer_id_b).unwrap().score().aggregate_score(),
        config.max_score,
        "B's score remains max after Fatal penalty attempt"
    );
}
```

Result

The test confirms that all conditions described in the scenario hold true:

- The rotated-out validator is no longer part of the active committee.
- The `is_trusted` flag remains set to `true` after the epoch transition.
- The peer retains the maximum reputation score.
- Applying `Penalty::Fatal` has no effect, demonstrating penalty immunity.

These results validate that trusted status is not revoked during epoch rotation and that a former committee member preserves privileged network status beyond its intended lifetime.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (7.5)

Recommendation

It is recommended to add a `make_untrusted()` method to the Peer struct that atomically resets `is_trusted` to `false` and resets the score to `Score::default()`, and to call this method inside `AllPeers::new_epoch()` for every peer present in the peers map whose `PeerId` is absent from the incoming committee list.

Remediation Comment

SOLVED: The **ParFin team** has solved this issue through the removal of the `is_trusted` flag from peers excluded from the new committee and the resetting of their score value.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/328987110ce1dfddec0207a01015b4e9a9e97ec3

7.3 UNVALIDATED SELF-REPORTED NODERECORD TIMESTAMPS ENABLE COMMITTEE PEER EVICTION

// HIGH

Description

The `timestamp` field inside a Kademlia `NodeRecord` is fully self-reported by the publishing node and accepted without any validation or clamping when received from remote peers. `NodeRecord::build` sets the field to the local `now()` when a node announces itself, but nothing prevents a remote peer from publishing a record with an arbitrary `u64` value. This single missing check enables two compound attacks.

Sub-finding A — DHT Record Freeze

`is_newer_record` gates whether an incoming record replaces the locally cached one using a strict less-than timestamp comparison. If a peer publishes its own `NodeRecord` with `timestamp = u64::MAX`, no future legitimate update can satisfy `u64::MAX < incoming.timestamp`, so the comparison permanently returns `false`. The poisoned record is frozen in the Kademlia store of every node that received it. Additionally, `process_kad_query_result` also selects the highest-timestamp result as the "best" during DHT queries, so the poisoned record wins all future query competitions indefinitely.

Sub-finding B — Committee Peer Permanent Isolation via Sybil Timestamp Flood

`cleanup_known_peers` enforces the `MAX_KNOWN_PEERS = 2000` limit by sorting `known_peers` ascending by the `NetworkInfo.timestamp` field and evicting the oldest entries first. An attacker can generate 2000+ BLS keypairs (trivially fast, no registration required), self-sign a `NodeRecord` per keypair with `timestamp = u64::MAX`, and send each via Kademlia PUT (no rate limiting exists, and valid records incur no penalty). All 2000 slots are then permanently occupied by attacker entries sitting at the tail of every ascending sort, making them immune to eviction.

The consequence surfaces across the full epoch-start sequence, which the orchestrator executes in strict order for every new epoch:

Copy Code

```
Step 1 — add_bootstrap_peers(committee)
           → NetworkCommand::AddBootstrapPeers
           → peer_manager.add_known_peer()     ← inserts committee member
           → cleanup_known_peers() triggered   ← member immediately evicted
                                                   (lowest timestamp, first candidate)

Step 2 — new_epoch(committee_keys)
           → NetworkCommand::NewEpoch
           → peer_manager.new_epoch()
           → looks up each member in known_peers
           → member absent → NOT added to current_committee

Step 3 — dial_peer_bls(bls_pubkey)
           → handle.dial_by_bls(bls_key)
           → NetworkCommand::DialBls
           → peer_manager.auth_to_peer(bls_key)  ← looks up known_peers
           → member absent → Err(PeerMissing)
           → retried with exponential backoff, gives up after 10 attempts
```

Code Location

Code of `is_newer_record` function from `crates/consensus/network/src/consensus.rs` file:

```rust
fn is_newer_record(&mut self, record: &kad::Record) -> bool {
    let store = self.swarm.behaviour_mut().kademlia.store_mut();

    if let Some(existing) = store.get(&record.key) {
        match (
            try_decode::<NodeRecord>(&existing.value),
            try_decode::<NodeRecord>(&record.value),
        ) {
            (Ok(existing_record), Ok(new_record)) => {
                // return true if the new record is newer
                existing_record.info.timestamp < new_record.info.timestamp
            }
            _ => false,
        }
```

Code of `cleanup_known_peers` function from `crates/consensus/network/src/peers/manager.rs` file:

```rust
fn cleanup_known_peers(&mut self) {
    if self.known_peers.len() <= MAX_KNOWN_PEERS {
        return;
    }

    // Find the oldest entries to remove based on timestamp
    let entries_to_remove = self.known_peers.len() - MAX_KNOWN_PEERS;

    // Collect (bls_key, timestamp) pairs and sort by timestamp
    let mut entries: Vec<_> = self
        .known_peers
        .iter()
        .map(|(k, v)| (*k, v.timestamp, v.pubkey.clone()))
        .collect();
    entries.sort_by_key(|(_, ts, _)| *ts);

    // Remove the oldest entries (but never remove validators)
    let mut removed = 0;
    for (bls_key, _, pubkey) in entries.into_iter().take(entries_to_remove * 2) {
```

Impact

Both sub-findings share the same root cause and their effects compound into a coherent attack surface against peer discovery, committee recognition, and consensus connectivity.

At the DHT level (Sub-finding A), any peer can publish its own `NodeRecord` with `timestamp = u64::MAX`, permanently poisoning that key across the entire network: no legitimate address update can displace it, and any node querying the DHT receives the frozen record indefinitely. This degrades the reliability of address discovery for individual keys.

At the `known_peers` level (Sub-finding B), the same primitive becomes a systematic weapon against the complete epoch-start bootstrap sequence. An attacker with a single P2P connection and 2000 self-generated BLS keypairs can permanently occupy all 2000 `known_peers` slots. The attack operates silently across two critical epoch-start paths simultaneously: `new_epoch` never adds the evicted

committee member to `current_committee` (so the node does not recognize them as a validator, they receive no trusted-peer status, and they are not penalty-immune), and `dial_by_bls` resolves to `auth_to_peer` which returns `PeerMissing` for every dial attempt. The orchestrator retries the dial with exponential backoff, but after 10 failed attempts with at least one other peer connected, it **permanently abandons** the dial to that committee member for the rest of the epoch. No recovery mechanism exists within the epoch: the only path to re-learning the address is via DHT, which the attacker continues to block.

The combined result is a **targeted, persistent isolation** of the victim node from specific committee members, achievable by any external actor with a single P2P connection, that causes the victim to silently fail to meet the connectivity requirements for consensus participation without any detectable protocol-level error.

Proof of Concept

Scenario

The test reproduces the complete attack sequence in three phases:

1. **Flood phase:** `MAX_KNOWN_PEERS` Sybil entries are inserted with `timestamp = u64::MAX`, filling the table to capacity.
2. **Epoch bootstrap phase:** The legitimate committee member's `NodeRecord` is inserted (simulating `add_bootstrap_peers`). `cleanup_known_peers` fires and evicts it because its `now()` timestamp is less than every Sybil entry.
3. **Impact phase:** `new_epoch` is called with the committee's BLS key set. Because the member is absent from `known_peers`, its `PeerId` cannot be resolved and it is never registered in `current_committee`.

Test Code

Copy Code

```
async fn test_sec_net03_known_peers_flood_evicts_committee_member() {
    let mut peer_manager = create_test_peer_manager(None);
    let mut rng = StdRng::from_seed([42; 32]);

    // ------------------------------------------------------------------------
    // Phase 1 — flood known_peers with MAX_KNOWN_PEERS Sybil entries.
    //
    // ------------------------------------------------------------------------
    for _ in 0..MAX_KNOWN_PEERS {
        let attacker_bls = *BlsKeypair::generate(&mut rng).public();
        let attacker_net: NetworkPublicKey = NetworkKeypair::generate_ed25519().public().into();
        peer_manager.add_known_peer(
            attacker_bls,
            NetworkInfo {
                pubkey:     attacker_net,
                multiaddrs: vec![create_multiaddr(None)],
                timestamp:  u64::MAX, // far-future: entry sits at the tail of every eviction sort
            },
        );
    }

    assert_eq!(
        peer_manager.known_peers.len(),
        MAX_KNOWN_PEERS,
        "Precondition: known_peers must be at capacity before the attack's second phase"
```

```
    );

    // ------------------------------------------------------------------------
    // Phase 2 — a legitimate committee member publishes its NodeRecord.
    //
    // This simulates process_kad_put_request accepting a valid, correctly
    // timed NodeRecord from an honest new committee member (timestamp = now()).
    // ------------------------------------------------------------------------
    let committee_bls = *BlsKeypair::generate(&mut rng).public();
    let committee_net: NetworkPublicKey = NetworkKeypair::generate_ed25519().public().into();
    let committee_peer_id: PeerId = committee_net.clone().into();
    let committee_addr = create_multiaddr(None);

    peer_manager.add_known_peer(
        committee_bls,
        NetworkInfo {
            pubkey:     committee_net,
            multiaddrs: vec![committee_addr.clone()],
            timestamp:  now(), // honest, current timestamp
        },
    );


    // Assert the buggy eviction DID occur (vulnerability confirmed).
    assert!(
        !peer_manager.known_peers.contains_key(&committee_bls),
        "Bug B not reproduced: committee member was NOT evicted. \
         The vulnerability may have been fixed (timestamp validation now in place)."
    );

    // ------------------------------------------------------------------------
    // Phase 3 — demonstrate the security impact at epoch transition.
    //
    // ------------------------------------------------------------------------
    let mut committee_keys = HashSet::new();
    committee_keys.insert(committee_bls);
    peer_manager.new_epoch(committee_keys);

    // After new_epoch the committee member should be in current_committee and
    // recognised as a validator.  Because their known_peers entry was evicted,
    // new_epoch silently skipped them and they are NOT in current_committee.
    assert!(
        !peer_manager.is_peer_validator(&committee_peer_id),
        "Bug B impact not reproduced: the committee member IS recognised as a validator. \
         The vulnerability may have been fixed."
    );
}
```

Result

Running the test against the unpatched codebase produces the following observations, all verified by the test assertions:

- **Eviction confirmed**: After the committee member's `NodeRecord` is inserted, `known_peers.contains_key(&committee_bls)` returns `false`. The entry was evicted by `cleanup_known_peers` in the same call frame.
- **Committee registration failure:** After `new_epoch(committee_keys)` is called, `peer_manager.is_peer_validator(&committee_peer_id)` returns `false`, confirming that the victim node does not recognise the targeted node as a current validator.
- **Attacker entries unaffected:** All 2,000 Sybil entries remain in `known_peers`, confirming that the eviction is asymmetric: entries with `timestamp = u64::MAX` are never selected as eviction candidates.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (7.5)

Recommendation

It is recommended to validate the `timestamp` field upon receipt in `process_kad_put_request`, before calling `is_newer_record` or `add_known_peer`: reject any `NodeRecord` whose timestamp deviates from the local clock by more than a small configurable skew window (e.g., ±5 minutes), returning early with a `Penalty` for the sender.

Additionally, `cleanup_known_peers` should track a locally assigned `inserted_at` timestamp (using `Instant::now()` recorded at the time `add_known_peer` is called) and use that for eviction ordering, fully decoupling eviction priority from any externally supplied field. As a defense-in-depth measure, a per-connection rate limit on Kademlia PUT requests should be enforced to slow Sybil flooding independently of timestamp validation.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by introducing timestamp validation on inbound Kademlia records and by replacing the externally-supplied record timestamp with a locally-assigned insertion time for peer eviction ordering.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/637fbf68da3925372b9d63b6950cc8fc3689547c

7.4 GOSSIP REJECTION PENALIZES HONEST RELAY PEER INSTEAD OF MALICIOUS AUTHOR

// MEDIUM

Description

When a `gossip` message fails validation, the penalty is applied to `propagation_source` (the peer that relayed the message) rather than to `message.source`, the original author. This misattribution occurs independently at two different layers of the network stack.

- **Layer 1 — Gossip authorization (`process_gossip_event`):** When `verify_gossip()` rejects a message, `Penalty::Fatal` is applied directly to `propagation_source`. The original author is only logged but never penalized.
- **Layer 2 — Application processing (primary and worker `process_gossip`):** When `verify_gossip()` accepts a message, the network layer resolves propagation source's PeerId to a BLS key via `peer_to_bls()` and propagates it downstream as the identity associated with the message. If deeper content validation then fails inside `request_handler.process_gossip()`, the penalty is applied to that resolved BLS key (again the relay's identity, not the author's).

Both paths therefore converge on the same misattribution: the relay absorbs all reputation damage regardless of which validation stage catches the invalid content.

Code Location

Code of `process_gossip_event` from crates/consensus/network/src/consensus.rs file:

Copy Code

```
854 if valid {
855     // We should not be able to receive a message from an unknown peer so this
856     // should always work.
857     if let Some(bls) =
858         self.swarm.behaviour().peer_manager.peer_to_bls(&propagation_source)
859     {
860         // forward gossip to handler
861         if let Err(e) =
862             self.event_stream.try_send(NetworkEvent::Gossip(message, bls))
863         {
864             error!(target: "network", topics=?self.authorized_publishers.keys(), ?propagation_
865             // ignore failures at the epoch boundary
866             // During epoch change the event_stream receiver can be closed.
867             return Ok(());
868         }
869     }
870 } else {
871     let GossipMessage { source, topic, .. } = message;
872     warn!(
873         target: "network",
874         author = ?source,
875         ?topic,
876         ?propagation_source,
877         "received invalid gossip - applying fatal penalty to propagation source"
878     );
879     self.swarm
880         .behaviour_mut()
881         .peer_manager
882         .process_penalty(propagation_source, Penalty::Fatal);
883 }
```

Code of `process_gossip` from crates/consensus/worker/src/network/mod.rs file (also applies to the **primary network**):

```
Copy Code
417 fn process_gossip(&self, msg: GossipMessage, propagation_source: BlsPublicKey) {
418     // clone for spawned tasks
419     let request_handler = self.request_handler.clone();
420     let network_handle = self.network_handle.clone();
421     let task_name = format!("process-gossip-{propagation_source}");
422     self.network_handle.get_task_spawner().spawn_task(task_name, async move {
423         if let Err(e) = request_handler.process_gossip(&msg).await {
424             warn!(target: "worker::network", ?e, "process_gossip");
425             if let Err(e) = request_handler.process_gossip(&msg).await {
426                 warn!(target: "worker::network", ?e, "process_gossip");
427                 // convert error into penalty to lower peer score
428                 if let Some(penalty) = e.into() {
429                     network_handle.report_penalty(propagation_source, penalty).await;
430                 }
431             }
432         }
433     });
434 }
```

Impact

The practical consequence depends on whether "**Stale trusted flag**" vulnerability is also present. When a Byzantine former committee member Alice retains `is_trusted = true` on node Bob after epoch rotation:

1. Bob accepts Alice's unauthorized gossip via the `peer_is_important()` fallback in `verify_gossip()` and forwards it to Bob's mesh neighbors (including victim Charles).

1. Bob's application layer identifies Alice as `propagation_source` and attempts to penalize it, but the same `is_trusted = true` flag makes `Peer::apply_penalty()` a no-op. Alice receives no effective penalty on Bob.

1. Charles receives the message with `propagation_source = Bob`. Depending on Charles's local trust state for Alice, either Layer 1 or Layer 2 fires, but in both cases the penalty lands on Bob, the honest relay.

The result is that the penalty is displaced from Alice to Bob at every node in the propagation path: Alice is shielded on stale-trust nodes, and on all other nodes it is penalized by the relay misattribution. Systematically repeating this across different relay paths progressively eliminates the victim's entire gossipsub mesh neighborhood, leading to consensus isolation without Alice ever accumulating a ban.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (6.7)

Recommendation

It is recommended to apply the penalty to the original message author (`message.source`) rather than to `propagation_source` at both layers.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by applying the penalties to the message author rather than propagator.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/292

7.5 MISSING PER-SENDER DEDUPLICATION IN EPOCH VOTE ALT-RECS COUNTER ENABLES SINGLE VALIDATOR TO FORCE RECOVERY PATH

// MEDIUM

Description

The `collect_epoch_votes` function maintains two parallel vote-tracking structures: a `HashSet<BlsPublicKey>` (`committee_keys`) for votes on the expected epoch hash, and a `HashMap<B256, usize>` (`alt_recs`) for votes on alternative epoch hashes. The `HashSet` correctly deduplicates by removing a key on first use and rejecting subsequent votes from the same sender. However, the `alt_recs` counter applies no such deduplication, it increments on every valid message received, regardless of whether the same `public_key` has already voted for that alternative hash.

The only validation applied to alt-recs votes is `committee.contains(public_key) && check_signature()`. These checks confirm the sender holds a valid BLS key in the current committee, but they do not prevent the same key from contributing multiple increments. A single malicious committee member can exploit this by signing one `EpochVote` for a fabricated `epoch_hash_evil` and publishing it `quorum` times over gossipsub, varying the `seqno` field in the gossip envelope on each transmission. Gossipsub computes `MessageId` over the full signed envelope including `seqno`; different `seqno` values produce different `MessageId`s, so each copy is treated as a distinct message and delivered to the application. The inner `EpochVote` payload (public key, epoch hash, and BLS signature) can be byte-for-byte identical across all copies.

The network layer provides no additional filtering. The `epoch_vote_topic` subscription is registered via `subscribe()` with `publishers: None`, which inserts `authorized_publishers["epoch_vote_topic"] = None`. Inside `verify_gossip`, the condition `auth.is_none()` evaluates to `true` for this topic, causing the function to accept gossip from any connected peer regardless of committee membership. All identity validation is therefore deferred to the application layer, where the deduplication gap exists.

Code Location

Code of `collect_epoch_votes` function from `crates/middleware/orchestrator/src/manager.rs` file:

Copy Code

```
946 let mut alt_recs: HashMap<B256, usize> = HashMap::default();
947 loop {
948     match tokio::time::timeout(timeout, rx.recv()).await {
949         Ok(Some((vote, vote_tx))) => {
950             if let Some(source) =
951                 Self::signed_by_committee(&epoch_rec.committee, &vote, epoch_hash)
952             {
953                 let _ = vote_tx.send(Ok(())); // If we lost this channel somehow then no big d
954                 if committee_keys.remove(&source) {
955                     sigs.push(vote.signature);
956                     if let Some(idx) = committee_index.get(&source) {
957                         signed_authorities.insert(*idx as u32);
958                     }
959                     if signed_authorities.len() >= quorum as u64 {
960                         reached_quorum = true;
961                         // We have quorum so just wait a sec longer for new certs then
```

```
                        // move on.
                        timeout = Duration::from_secs(1);
                    }
                    if signed_authorities.len() >= committee_size {
                        break;
                    }
                }
            } else {
                // Send an error back to punish the peer that sent a bad epoch vote.
                let err = if vote.epoch_hash != epoch_hash {
                    if epoch_rec.committee.contains(&vote.public_key)
                        && vote.check_signature()
                    {
                        // If we got a valid vote on another epoch record track that and break
                        // This will cause us to query the record and cert from a peer.
                        let votes = *alt_recs.get(&vote.epoch_hash).unwrap_or(&0);
                        if votes + 1 >= quorum {
                            error!(
                                target: "epoch-manager",
                                "Reached quorum on epoch record {} instead of {}.",
                                vote.epoch_hash,
                                epoch_hash,
                            );
                            let _ = vote_tx.send(Err(HeaderError::InvalidHeaderDigest));
                            break;
                        }
                        // Rayls: limit alt_recs to prevent unbounded growth
                        const MAX_ALT_RECS: usize = 100;
                        if alt_recs.len() < MAX_ALT_RECS || alt_recs.contains_key(&vote.epoch_
                            alt_recs.insert(vote.epoch_hash, votes + 1);
                        }
                    }
                    HeaderError::InvalidHeaderDigest
```

Impact

- **Liveness disruption of epoch certificate collection**

A single active committee member can force any victim node into the `collect_epoch_votes` recovery path within seconds, without waiting for any organic network failure. The break triggered by `votes + 1 >= quorum` sets `reached_quorum = false` and diverts execution to the unprotected peer-recovery block, bypassing the normal certificate aggregation path entirely.

- **Entry trigger for epoch record poisoning (FIND-021 chain).**

The recovery block at line 1055 requests an `(EpochRecord, EpochCertificate)` pair from a connected peer via `request_epoch_cert(epoch=N, hash=None)` and validates the response using only `verify_with_cert`, a self-referential check that accepts any record whose embedded committee matches its own certificate. A Byzantine peer can respond with a forged `EpochRecord` containing an attacker-controlled committee and `next_committee`. This record is written to `consensus_db` via `save_epoch_record_with_cert`, permanently overwriting `EpochRecords[N]`.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:M/D:N/Y:N (6.3)

Recommendation

It is recommended to replace the plain `HashMap<B256, usize>` counter in `alt_recs` with a `HashMap<B256, HashSet<BlsPublicKey>>` structure that records which distinct public keys have voted for each alternative epoch hash, counting only the first vote per unique key, mirroring the deduplication already applied to the primary vote path.

Remediation Comment

SOLVED: The **ParFin team** partially addressed this issue by replacing the plain `HashMap<B256, usize>` counter in `alt_recs` with a generic `VotesAggregator<EpochVote>` that internally tracks seen voters via a `HashSet<AuthorityIdentifier>`, ensuring that duplicate votes from the same BLS public key are rejected.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/266

7.6 SELF-REFERENTIAL EPOCH RECORD VALIDATION ENABLES FAKE EPOCHRECORD INJECTION VIA RECOVERY PATH

// MEDIUM

Description

At the start of every epoch N+1, `collect_epoch_votes(epoch_rec[N])` is spawned as a background task to collect BLS signatures from committee members and build an `EpochCertificate` for the just-closed epoch N.

The function collects votes via gossip with a 5-second `tokio::time::timeout` per iteration and a maximum of 13 iterations. If quorum (`(2n/3) + 1`) is not reached, `reached_quorum` remains `false` and execution falls into a recovery block that requests a `(EpochRecord, EpochCertificate)` pair from a connected peer.

The recovery block of `collect_epoch_votes` applies a single validation to the peer-supplied pair (`crates/middleware/orchestrator/src/manager.rs` file):

Copy Code

```
1060    // Try to recover by downloading the epoch record and cert from a peer.
1061    let mut got_epoch_record = false;
1062    for _ in 0..3 {
1063        // Request by epoch number in case we had a bad hash...
1064        match primary_network.request_epoch_cert(Some(epoch_rec.epoch), None).await {
1065            Ok((new_epoch_rec, cert)) => {
1066                if new_epoch_rec.verify_with_cert(&cert) {
1067                    let new_epoch_hash = new_epoch_rec.digest();
1068                    // Humm, we got another epoch record than the one we expected...
1069                    // The network came to quorum on this one so lets go with it...
1070                    if new_epoch_hash != epoch_hash {
1071                        warn!(
1072                            target: "epoch-manager",
1073                            "Over wrote expected epoch record {epoch_hash} with verified epoch r
1074                        );
1075                        if let Err(e) = consensus_db.save_epoch_record_with_cert(&new_epoch_rec,
1076                            error!(
1077                                target: "epoch-manager",
1078                                "failed to save epoch record with cert: {e}",
1079                            );
1080                        }
1081                    } else {
1082                        info!(
1083                            target: "epoch-manager",
1084                            "retrieved cert for epoch {new_epoch_hash} from a peer",
1085                        );
1086                        let _ = consensus_db.insert::<EpochCerts>(&new_epoch_hash, &cert);
1087                    }
1088                    got_epoch_record = true;
1089                    break;
1090                }
1091            }
1092            Err(err) => error!(
1093                target: "epoch-manager",
1094                "failed to retrieve epoch from a peer {epoch_hash}: {err}",
1095            ),
1096        }
1097    }
```

The branch logic is **counterintuitive:** the `else` arm (same hash) is the benign case (only a missing cert is stored). The `if new_epoch_hash != epoch_hash` arm is the **active injection branch**: when the peer

returns a record with a *different* hash, the code assumes the network reached quorum on an alternative record and **overwrites** `EpochRecords[N]` **entirely** with whatever the peer sent, as long as `verify_with_cert` returns `true`.

The root cause is that `verify_with_cert` resolves the signing committee from the `committee` field **embedded in the record being verified**, not from any trusted external anchor. There is no cross-check against the previous epoch's `next_committee` from the local DB, `ConsensusRegistry::validators_for_epoch(N)` from the on-chain state, or `parent_hash` chain continuity.

An attacker who controls any peer reachable via `send_request_any` can fabricate:

- `fake_epoch_rec_N` with `committee = [attacker_bls_pubkey]` and `next_committee = [attacker_bls_pubkey]`
- `fake_cert_N` signed with the attacker's private key, with `signed_authorities = {0}` (yielding `super_quorum(1) = 1`)

Contrast with the protected path:
The equivalent function `collect_epoch_records` in `crates/consensus/state-sync/src/epoch.rs` applies three conditions before accepting any downloaded pair:

```
// Verify the epoch has the expected parent and committee and is signed by that committee.
if parent_hash == epoch_rec.parent_hash          // 1. chain continuity
    && epoch_committee_valid(&epoch_rec, &committee) // 2. committee matches prev.next_committ
    && epoch_rec.verify_with_cert(&cert)         // 3. cert signed by that committee
{
    let epoch_hash = epoch_rec.digest();
    if let Err(e) = db.save_epoch_record_with_cert(&epoch_rec, &cert) { ... }
}
```

Impact

- **Permanent poisoning of `consensus_db`:** `EpochRecords[N]` is overwritten. The early-exit guard in `collect_epoch_votes` now returns `Ok()` immediately on all future calls for epoch N, permanently preventing any legitimate recovery of that record.
- **Deferred crash and permanent restart loop:** When epoch N+1 closes, `write_epoch_record` is called to build the record for epoch N+1. It reads `EpochRecords[N]` as the previous epoch anchor and performs:

```
let committee_keys = engine.validators_for_epoch(N+1).await?; // real keys from ConsensusRegistry
let prev = self.consensus_db.get::<EpochRecords>(&N)?;        // ← poisoned record
if committee_keys != prev.next_committee {                     // real ≠ attacker keys → Err
    return Err(eyre!("Last epochs next committee not equal to this epochs committee!"));
}
```

This `Err` propagates through `run_epoch_transition_inner` → `run_epoch_transition` → `run_epoch` → `run_epochs` → `run()` → `launch_node` → `main.rs`, terminating the process with `std::process::exit(1)`.

- **Crash cycle on restart.** At the moment `write_epoch_record` fails, the transition checkpoint saved in DB is `Draining`, so the `ExecutionComplete` checkpoint is never reached. On every subsequent

restart, `recover_partial_transition` loads the `Draining` checkpoint and evaluates `execution_done` via `tip_state.epoch > epoch`, which resolves to `N+1 > N+1 = false`. The checkpoint is cleared and the node re-runs epoch N+1 from consensus. When it re-reaches the N+1 → N+2 boundary, `write_epoch_record` reads the same poisoned `EpochRecords[N]`, fails identically, and exits again. **The node cannot complete the N+1 → N+2 epoch transition on any restart.** Manual removal of the poisoned `EpochRecords[N]` entry from DB is required to break the cycle.

- **Cascading propagation.** While still running epoch N+1, the victim actively serves `(fake_rec_N, fake_cert_N)` in response to any `request_epoch_cert(epoch=N)` from the network, because `retrieve_epoch_record` reads from the same poisoned DB entry.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:H/I:M/D:N/Y:N (5.9)

Recommendation

It is recommended to remove the overwrite branch from the `collect_epoch_votes` recovery block. The node already holds its own locally computed `epoch_rec[N]`, derived from `engine.validators_for_epoch()` and written to the consensus DB by `write_epoch_record` before `collect_epoch_votes` is ever called. What the recovery path is actually missing is the `EpochCertificate` for that record, not the record itself. Therefore, the only peer response that should be accepted is one whose `epoch_hash` matches the locally computed hash.

If the alternative-hash case must be preserved, it is recommended to apply the same three-condition validation already enforced by `collect_epoch_records`, anchoring every check to data that the node already holds in its local consensus DB.

Remediation Comment

SOLVED: The **ParFin team** addressed this issue by applying the same three-condition validation already enforced by the protected `collect_epoch_records` sync path to the `collect_epoch_votes` recovery block: `verify_with_cert`, `epoch_committee_valid` against the locally-known committee snapshot, and `parent_hash` chain continuity.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/283

7.7 UNGUARDED SLICE IN FUNCTION TRIGGERS REMOTE PANIC VIA MALFORMED TXN GOSSIP

// MEDIUM

Description

The `submit_batch_if_mine` function performs an unconditional 8-byte slice index on the first element of the incoming transaction list without first validating that the element is long enough. Any connected network peer, including non-committee external nodes, which are explicitly expected to publish to the `GOSSIP_TOPIC_TXN` topic, can send a `WorkerGossip::Txn` payload whose first inner vector is fewer than 8 bytes. When this payload reaches `submit_batch_if_mine`, the slice operation panics.

The gossip topic is subscribed with `publishers: None` at startup:

- **middleware/orchestrator/src/manager.rs** (subscription with no publisher restrictions)

Copy Code

```
1760 let txn_topic = LibP2pConfig::worker_txn_topic();
1761 info!(target: "epoch-manager::gossipsub", ?txn_topic, "subscribing to worker txn topic");
1762 network_handle.inner_handle().subscribe(txn_topic.clone()).await?;
```

- **consensus/network/src/types.rs** (any publisher valid)

Copy Code

```
417 pub async fn subscribe(&self, topic: String) -> NetworkResult<bool> {
418     let (reply, already_subscribed) = oneshot::channel();
419     self.sender.send(NetworkCommand::Subscribe { topic, publishers: None, reply }).await?;
```

This means `verify_gossip` resolves `auth_config` to `Some(None)` for the txn topic, causing `auth.is_none()` to short-circuit to `true` and accepting any connected peer as a valid publisher.

Copy Code

```
1095 let auth_config = self.authorized_publishers.get(topic.as_str());
1096 if auth_config.is_none() {
1097     if let Some(source_id) = gossip.source {
1098         if self.swarm.behaviour().peer_manager.peer_is_important(&source_id) {
1099             trace!(
1100                 target: "network",
1101                 ?topic,
1102                 ?source_id,
1103                 "accepting gossip from validator during startup grace period"
1104             );
1105             return GossipAcceptance::Accept;
1106         }
1107     }
1108 }
```

The panic occurs inside the **gossip-processing task**, which is spawned asynchronously after the network layer has already accepted and propagated the message. As a result, the malformed message is forwarded to all nodes in the gossipsub mesh before the panic is triggered. Each receiving node spawns a

task that panics independently, producing **network-wide panic amplification from a single injected message**.

The panic does not crash the node because the task is spawned as a **non-critical (`TaskKind::Doomed`) task**, but it terminates the processing task immediately. Additionally, the panic bypasses the normal error-handling path in `process_gossip`, meaning that:

- No penalty is applied to the sending peer.
- No warning log is emitted.
- The attacker can repeatedly trigger the condition without reputation impact.

Code Location

Code of `submit_batch_if_mine` function from **crates/consensus/worker/src/batch-validator/src/validator.rs** file:

Copy Code

```
102 fn submit_batch_if_mine(
103     &self,
104     txs_bytes: &[Vec<u8>],
105     committee_size: u64,
106     committee_slot: u64,
107 ) {
108     if let Some(tx_pool) = &self.tx_pool {
109         // loop to check if the batch is for this validator because some txns may be errors
110         if let Some(tx) = txs_bytes.iter().next() {
111             let mut bytes = [0_u8; 8];
112             bytes.copy_from_slice(&tx[0..8]);
113             if (u64::from_le_bytes(bytes) % committee_size) != committee_slot {
114                 return;
115             }
116         }
```

Impact

A malicious peer can repeatedly broadcast malformed transactions that trigger panics in the gossip processing pipeline across all committee nodes.
This results in:

- **Persistent task churn** within the Tokio executor as gossip tasks repeatedly panic.
- **Executor resource contention**, reducing capacity available for legitimate transaction processing.
- **Network-wide amplification**, since a single injected message propagates to all committee nodes before the panic occurs.
- **No automatic mitigation**, as the attacker receives no penalty and can repeat the attack indefinitely.

The issue does not cause node crashes or consensus failures, but it can **significantly degrade transaction ingestion throughput across the validator committee**.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

Recommendation

It is recommended to add an explicit minimum-length guard before the slice index operation in `submit_batch_if_mine`, returning early when the invariant is violated:

```
if tx.len() < 8 {
    warn!(target: "worker:validator", "received gossip txn shorter than 8 bytes, skipping");
    return;
}
```

Additionally, it is recommended to refactor `submit_batch_if_mine` to return a `Result` instead of being an infallible function, so that input validation failures propagate through the standard error-handling pipeline and allow the penalty mechanism to penalize peers who send malformed transaction gossip.

Remediation Comment

SOLVED: The **ParFin team** solved this issue through graceful handling of the potential panic.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/00a3ced7e92d99858a28c44a067b5b01314a855d

7.8 EPOCH TRANSITION CLEARS NON-VALIDATOR IP BANS AND PERMANENTLY DESYNCHRONIZES BAN COUNTER

// MEDIUM

Description

During `new_epoch()`, the node calls `remove_validator_ip()` for every committee member that is added to the new epoch. The intent of this call is to prevent a validator's own IP from being blocked when the validator reconnects at the start of a new epoch. However, the implementation unconditionally deletes the entire `banned_peers_by_ip` entry for each of the validator's IPs using `remove_entry()`, without accounting for how many other (non-validator) peers were banned at that IP.

This creates two compound defects:

- Defect 1 - **Non-validator IP ban erasure (ban evasion)**: If an attacker shares an IP address with a committee validator (a realistic condition in cloud environments with shared NAT gateways) the epoch transition automatically and silently lifts the IP ban on the attacker. The attacker takes no action; the unban is a side effect of the legitimate validator joining a new committee. This repeats at every epoch, giving the attacker a persistent, free reconnection window after each epoch boundary.
- Defect 2 - `total` **counter desynchronization**: `remove_validator_ip()` decrements the `total` counter by exactly 1 per removed IP entry, regardless of the `count` value stored in that entry (which may be ≥ 2 if multiple peers from the same IP were banned). After the removal, `total` over-counts the actual number of banned peers. This corrupts `prune_banned_peers()`, which uses `total()` to calculate how many peers to evict: with an inflated `total`, spurious excess is computed, and the pruning loop runs but finds no matching peers leading to a permanently incorrect state that accumulates across epochs.

Code Location

Code of `remove_validator_ip` function from `crates/consensus/network/src/peers/banned.rs` file:

Copy Code

```
75  pub(super) fn remove_validator_ip(
76      &mut self,
77      peer_id: &PeerId,
78      ip_addresses: impl Iterator<Item = IpAddr>,
79  ) {
80      debug!(target: "peer-manager", ips=?self.banned_peers_by_ip, "filtering banned ips for val
81      for ip in ip_addresses {
82          debug!(target: "peer-manager", ?ip, "removing banned ip for validator");
83          if let Some((_, _)) = self.banned_peers_by_ip.remove_entry(&ip) {
84              warn!(target: "peer-manager", ?peer_id, "removed banned ip address associated with
85              self.total = self.total.saturating_sub(1);
86          }
87      }
88  }
```

Impact

An external actor co-located at the same IP as any committee validator obtains **persistent, automatic IP ban evasion** that renews at every epoch boundary without requiring any action from the attacker. The ban accounting system's correctness degrades monotonically across epochs due to the `total` counter drift. Over many epochs, the accumulated desynchronization renders the IP-level ban system unreliable as a security control.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (5.0)

Recommendation

It is recommended to replace the `remove_entry()` call in `remove_validator_ip()` with a scoped, per-validator contribution tracking approach: instead of deleting the entire IP key, track separately how many banned entries are attributable to validators versus non-validators, decrementing only the validator's own contribution (for example, track validator IPs in a separate set and exempt them at connection time without modifying the shared count).

If full separation is not desired, a simpler fix is to skip the `remove_entry()` call entirely and instead rely on the existing per-peer validator exemption in `peer_banned()`, which already prevents committee members from being treated as banned regardless of the IP state. Additionally, `total` should only be decremented by the actual stored `count` of the removed entry (not a fixed `1`) to preserve the invariant `total == sum(banned_peers_by_ip.values())`.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by eliminating both the unconditional `remove_entry()` call that silently lifted IP bans on epoch transitions and the `total` counter desynchronization caused by fixed-decrement removals that ignored the actual stored count.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/b77e45c9798265c56dcab0742aa872d7dc8ee037

7.9 UNVERIFIED BLS SIGNATURES ON PEER-FETCHED CONSENSUS ENABLE BYZANTINE CACHE POISONING

// MEDIUM

Description

The function `get_consensus_header_no_notify`, from State Sync module, fetches a `ConsensusHeader` from any connected peer using `send_request_any` and validates the response with a single check: `header.digest() == hash`. This check is necessary but not sufficient, because `CommittedSubDag::digest()` explicitly excludes BLS signature bytes from its hash commitment:

```
ConsensusHeader.digest()
  └── sub_dag.digest()          ← CommittedSubDag::digest()
        └── cert.digest()       ← Certificate::digest()
              └── header.digest()  ← only author, round, epoch, payload, parents
                                   ← signature_verification_state: NO
                                   ← signed_authorities: NO
```

`Certificate::digest()` is implemented as `self.header.digest()`,it hashes only the `Header` fields (author, round, epoch, payload, parents, latest_execution_block). The fields `signature_verification_state` and `signed_authorities`, which together constitute the BLS quorum proof, are not part of the hash at any level of the commitment chain: `ConsensusHeader.digest() → CommittedSubDag::digest() → cert.digest() = header.digest()`. A Byzantine validator that participated in consensus already holds the complete legitimate `ConsensusHeader` with valid signatures. It can strip the `signature_verification_state` from every certificate in the `sub_dag` (replacing them with zeroed or fabricated bytes) and the resulting header still produces the same `digest()` as the original. The hash check `header.digest() == hash` passes unconditionally, because it cannot detect any modification to the signature fields.

The received header is then written directly to persistent storage by `store_consensus_header_in_cache` without any call to `verify_certificates()`. The `ConsensusHeader::verify_certificates` method (which performs full BLS signature verification against the epoch committee) exists and is correct, but is never invoked in this code path.

The poisoned entry propagates downstream through the execution pipeline without any further signature check: `catch_up_consensus_from_to` reads headers from `ConsensusBlocksCache` and verifies only digest chain integrity (hash-to-hash), then forwards them via `consensus_bus.consensus_header().send()`. The subscriber's `handle_consensus_header` writes the poisoned header to the canonical `ConsensusBlocks` table via `save_consensus`, and stores the individual invalid-signature certificates via `write()` / `write_all()`. Once in the canonical store, the node serves these poisoned headers to other syncing nodes that request the same block hash, propagating the invalid data laterally across the observer/inactive peer set.

Code Location

Code of `get_consensus_header_no_notify` function from `crates/consensus/state-sync/src/consensus.rs` file:

```
async fn get_consensus_header_no_notify<DB: ReDatabase>(
    hash: B256,
    config: &ConsensusConfig<DB>,
    network: &PrimaryNetworkHandle,
) -> Option<(ConsensusHeader, B256)> {
    let db = config.node_storage();
    if let Some(block) = db.get_consensus_by_hash(hash) {
        return Some((block.clone(), block.parent_hash));
    }
    // request consensus from any peer
    if let Ok(header) = network.request_consensus(None, Some(hash)).await {
        let parent = header.parent_hash;
        store_consensus_header_in_cache(db, &header);
        Some((header, parent))
    } else {
        None
    }
}
```

Code of `digest` function from `crates/infrastructure/types/src/primary/output.rs` file:

```
impl Hash<{ crypto::DIGEST_LENGTH }> for CommittedSubDag {
    type TypedDigest = ConsensusDigest;

    fn digest(&self) -> ConsensusDigest {
        let mut hasher = crypto::DefaultHashFunction::new();
        // Instead of hashing serialized CommittedSubDag, hash the certificate digests instead
        // Signatures in the certificates are not part of the commitment.
        for cert in &self.certificates {
            hasher.update(cert.digest().as_ref());
        }
        hasher.update(self.leader.digest().as_ref());
        // skip reputation for stable hashes
        hasher.update(encode(&self.commit_timestamp).as_ref());
        ConsensusDigest(Digest { digest: hasher.finalize().into() })
    }
```

Code of `digest` function from `crates/infrastructure/types/src/primary/certificate.rs` file:

```
impl Hash<{ crypto::DIGEST_LENGTH }> for Certificate {
    type TypedDigest = CertificateDigest;

    fn digest(&self) -> CertificateDigest {
        CertificateDigest(Digest { digest: self.header.digest().into() })
    }
}
```

Impact

A Byzantine validator connected as a peer can inject headers with invalid BLS signatures into the local DB of any `CvvInactive` or `Observer` node. The poisoned entries survive node restart and are eventually promoted to the canonical `ConsensusBlocks` table.

The concrete impacts are:

- **Lateral propagation**: other syncing nodes request the same block hash and receive the poisoned version from the victim's canonical store, spreading invalid-signature data across the full set of non-active peers.
- **Consensus rejoin disruption**: if the victim later transitions to `CvvActive`, the individual certificates with invalid signatures stored via `write_all()` are referenced as parents in future headers. Honest peers running `cert_validator.validate_and_verify` reject those parent references, preventing the rejoining node from successfully participating in consensus rounds.

Note that the certificate content (author, payload, parent references) is hash-committed and cannot be altered by the Byzantine peer, so transaction execution correctness is not directly affected. The practical impact is reduced by the peer rotation in `send_request_any`.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (5.0)

Recommendation

It is recommended to introduce certificate signature verification at the point where cached headers are promoted to the canonical `ConsensusBlocks` table.

Verification should call `consensus_header.verify_certificates(committee)` using the committee retrieved for `leader_epoch()` from the `EpochRecords` table. If `get_committee_keys` returns `None` for that epoch (indicating the epoch record has not yet been persisted) the header must be held back and not forwarded to the execution pipeline until the committee becomes available, rather than silently accepting it. Headers that fail certificate verification must be discarded and must not be written to `ConsensusBlocks` or the certificate store. Since `spawn_epoch_record_collector` and `spawn_track_recent_consensus` run concurrently with no ordering guarantee, an explicit coordination point between these two subsystems should be introduced: headers whose epoch records are not yet locally available should be queued and re-evaluated once the corresponding epoch record is confirmed present in the DB.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by adding BLS certificate verification via `verify_header_with_keys()` at every point where consensus headers received from peers are stored ensuring that headers with stripped or invalid signatures are discarded before being written to the cache or promoted to the canonical `ConsensusBlocks` table.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/273

7.10 PARKED BATCH DRAIN PATH BYPASSES DEDUPLICATION GUARD ENABLING STATE DIVERGENCE

// MEDIUM

Description

The `execute_consensus_output` function exposes two distinct paths through which batches reach execution, with asymmetric deduplication behaviour between them.

The normal in-order path applies a full deduplication guard before executing each batch: the `executed_batch_digests` set is locked, the digest is inserted, and execution proceeds only if the insertion succeeded (meaning the digest was not already present). If the digest was already in the set, the batch is skipped with a warning.

Copy Code

```
// skip duplicate batches that were already executed (e.g. via repropose)
{
    let mut digests = executed_batch_digests.lock();
    if !digests.insert(batch_digest) {
        warn!(
            target: "engine",
            ?batch_digest,
            output_number = output.number,
            "skipping duplicate batch already executed"
        );
        continue;
    }
}
```

The `drain_parked_for_authority` path, invoked after each successfully executed in-order batch to flush consecutive parked batches for the same authority, provides no equivalent mechanism. The function neither checks `executed_batch_digests` before calling `execute_payload` nor inserts the digest into the set after execution. The function signature does not receive `executed_batch_digests` as a parameter, confirming this is a structural omission rather than a conditional one.

At startup, `executed_batch_digests` is reconstructed from the last `DIGEST_RECONSTRUCTION_DEPTH = 1000` blocks by reading `mix_hash XOR parent_beacon_block_root` from each stored block header. Because `drain_parked_for_authority` invokes `execute_payload`, which produces standard chain blocks encoding the batch digest in `mix_hash`, those executions are present in the chain history and are correctly recovered into the set upon a node restart. A node that executed those same batches during the current session without restarting does not have those digests in its in-memory set, since the drain path never inserted them.

This asymmetry between the reconstructed state of a restarted node and the in-memory state of a continuously running node produces divergent deduplication decisions for any batch digest that subsequently arrives via the normal execution path.

Code Location

Code of `drain_parked_for_authority` function from `crates/middleware/processor/src/payload_builder.rs` file:

Copy Code

```
fn drain_parked_for_authority(
    batch_ordering: &BatchOrderingState,
    authority: Address,
    mut canonical_header: SealedHeader,
    executed_blocks: &mut Vec<ExecutedBlock>,
    reth_env: &RethEnv,
    gas_accumulator: &GasAccumulator,
    batch_tracker: &Option<Arc<BatchTracker>>,
    // ← executed_batch_digests is absent from the parameter list
) -> EngineResult<SealedHeader> {
    loop {
        let parked = {
            let mut seq_state = batch_ordering.lock();
            // ...
            match auth_state.parked.remove(&next_seq) {
                Some(parked) => {
                    auth_state.last_executed_seq = Some(next_seq);
                    parked
                    // ← no executed_batch_digests check before proceeding
                }
                None => break,
            }
        };
        // ...
```

Impact

The asymmetry between the deduplication state of a restarted node and a continuously running node produces divergent execution outcomes for identical consensus inputs.

A batch executed via `drain_parked_for_authority` is recorded on-chain through a standard block. A node restarted within the last 1000 blocks of that execution reconstructs the corresponding digest into `executed_batch_digests` and will skip any future appearance of that batch in the normal path. A continuously running node, lacking that digest in its in-memory set, will execute the batch again if the same digest appears in a subsequent `ConsensusOutput`.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (5.0)

Recommendation

It is recommended to pass `executed_batch_digests` as an explicit parameter to `drain_parked_for_authority` and apply the same deduplication guard present in the normal execution path: check whether the batch digest is already present in the set before invoking `execute_payload`, skip execution if it is, and insert the digest into the set after a successful `execute_payload` and block finalisation.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by ensuring the deduplication guard is uniformly applied across all execution paths, including the drain path.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/226

7.11 EPOCH COMMITTEE SELECTION SEED DERIVED UNILATERALLY FROM BOUNDARY LEADER

// LOW

Description

The randomness seed used to shuffle and select the next epoch's validator committee is derived exclusively from an artifact produced by a single node (the epoch boundary leader's BLS aggregate certificate signature) with no contribution from any other participant.
This is a **design-level problem**: as long as any single party proposes the epoch committee entropy unilaterally, a Byzantine instance of that party can try arbitrary variations of that entropy, adjusting which signatures are included in the aggregate, or any other degree of freedom it controls, until the resulting shuffle produces a committee favorable to it. The manipulation produces a protocol-valid certificate that is indistinguishable from an honest one.

The attack is conditional on two preconditions:

1. the number of active validators exceeds the committee size, so the shuffle produces a true selection rather than a full-set reordering neutralized by `new_committee.sort()`
2. the Byzantine validator is elected as the epoch boundary leader, which is probabilistic.

These preconditions limit current exploitability, particularly in a small fixed-committee deployment where all active validators are always selected.

Code Location

Code of `shuffle_new_committee` function from `crates/execution/evm/src/evm/block.rs` file:

Copy Code

```
fn shuffle_new_committee(&mut self, randomness: B256) -> RaylsRethResult<Vec<Address>> {
    let new_committee_size = self.next_committee_size()?;
    // ...reads active validators from ConsensusRegistry...

    // create seed from hashed bls agg signature
    let mut seed = [0; 32];
    seed.copy_from_slice(randomness.as_slice());  // [319] seed comes entirely from leader
    // used as deterministic randomness
    let mut rng = StdRng::from_seed(seed);        // [323] RNG seeded from single-party value

    // ...partition active / pending-exit validators...

    // simple Fisher-Yates shuffle
    for i in (1..validators_for_shuffle.len()).rev() {
        let j = rng.random_range(0..=i);          // [348] shuffle fully determined by leader's seed
        validators_for_shuffle.swap(i, j);
    }

    new_committee.truncate(new_committee_size);   // [358] selection is final
    Ok(new_committee)
}
```

Impact

When exploitable, a Byzantine epoch boundary leader can bias which validators are included in the next committee, increasing representation of colluding validators, excluding specific honest validators from participation and their associated epoch rewards, or influencing which `PendingExit` validators are retained in the committee despite having requested to leave.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

It is recommended to replace the single-party entropy source with a **multi-party contribution scheme**, ensuring no individual validator can unilaterally determine the shuffle seed.

Remediation Comment

RISK ACCEPTED: The **ParFin team** has accepted the risk associated with this finding.

7.12 THE CRITICAL HEALTHCHECK TASK MAY TRIGGER A FULL NODE SHUTDOWN

// LOW

Description

The healthcheck server is registered with `spawn_critical_task()`, classifying it as a **critical, non-drainable** `TaskKind::Doomed`) task inside the `TaskManager`. Its core accept loop uses the pattern `while let Ok(...) = listener.accept().await`, which silently exits the loop on any error returned by `accept()`. When a critical Doomed task returns `Ok(())` in normal (non-drain) mode, the `TaskManager` produces `TaskJoinError::CriticalExitOk("healthcheck")`, breaks its main loop, and initiates a full node shutdown.

The `accept()` system call can return errors under various OS-level conditions. The most relevant are `EMFILE` (per-process FD limit reached) and `ENFILE` (system-wide FD limit reached). Should either condition arise (whether through misconfiguration, operational overload, or an adversary exhausting the process FD pool through other services), the loop exits without recovery and the node shuts down.

Two design decisions significantly limit practical exploitability under standard deployments: the feature is **opt-in** (requires the `--healthcheck <PORT>` CLI flag or `HEALTHCHECK_TCP_PORT` environment variable, and is disabled by default), and the node explicitly calls `raise_fd_limit()` at startup to maximize the process soft limit. Furthermore, the sequential accept loop never holds more than one accepted FD at a time, making direct FD exhaustion through the healthcheck port alone infeasible.

Code Location

Code of `spawn` function from `crates/middleware/orchestrator/src/health.rs` file:

Copy Code

```rust
pub(crate) async fn spawn(task_spawner: TaskSpawner, port: u16) -> eyre::Result<SocketAddr> {
    // IMPORTANT: use firewall to protect this endpoint
    let addr: SocketAddr = ([0, 0, 0, 0], port).into();
    let listener = TcpListener::bind(addr).await?;
    let listen_on = listener.local_addr()?;
    info!(target: "epoch-manager", ?listen_on, "healthcheck listening");

    task_spawner.spawn_critical_task("healthcheck", async move {
        // minimal valid HTTP
        let response = b"HTTP/1.1 200 OK\r\nContent-Length: 2\r\n\r\nOK";

        // accept loop - no connection limits or timeouts
        while let Ok((mut socket, _)) = listener.accept().await {
            // write response, ignore errors (client disconnect, etc.)
            // then drop connection
            let _ = socket.write_all(response).await;
        }
    });

    Ok(listen_on)
}
```

Impact

When triggered, the impact is a complete, unrecoverable node shutdown initiated by the internal `TaskManager` shutdown sequence, indistinguishable from a crash. The severity is bounded by the conditions required to reach the vulnerable state: the flag must be explicitly enabled, the process FD limit must be exhausted (mitigated by `raise_fd_limit()` at startup), and the attack vector requires either environmental misconfiguration or compound resource exhaustion via other exposed surfaces. There is no cryptographic material compromise, no data corruption, and no consensus manipulation; the sole impact is **availability loss** (liveness failure).

Under a realistic production deployment with standard OS limits, the exploitability is low. Under constrained container environments with low hard FD limits, the risk increases.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (2.5)

Recommendation

It is recommended to:
1. **Replace while let Ok(...) with explicit error handling**: transient accept() errors (EMFILE, ENFILE, ENOBUFS, ENOMEM, ECONNABORTED) should be logged and retried after a short exponential back-off; only fatal errors indicating the listener itself is invalid should break the loop.
2. **Declassify the task**: a monitoring endpoint is not a node liveness prerequisite. Replace `spawn_critical_task()` with `spawn_task()`, or register it as `TaskKind::Drainable`. An unhealthy healthcheck should alert operators, not terminate the node.
3. **Add a connection rate limit or FD guard**: before calling `accept()`, optionally check current FD usage and introduce back-pressure rather than failing.

Remediation Comment

SOLVED: The **ParFin team** solved this issue through reclassification of the task from critical to normal.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/66ff1aa6ccfe9dfcc6da4cc6a48b61d21167a606

7.13 QUORUMWAITER REMAINING-TIME SUBTRACTION INVERSION PREVENTS PEER DRAIN TASK FROM SPAWNING

// LOW

Description

In `QuorumWaiter::verify_batch`, once quorum is reached the code attempts to compute the time remaining before the outer deadline, in order to spawn a bounded `quorum-remainder` drain task that lets the still-pending per-peer futures complete cleanly. The operands in the `saturating_sub` call are swapped, producing the **overtime** elapsed past the deadline instead of the **time budget remaining**:

Copy Code

```
// Current — computes overtime (elapsed − deadline)
let remaining_time = start_time.elapsed().saturating_sub(timeout);

// Correct — computes time remaining (deadline − elapsed)
let remaining_time = timeout.saturating_sub(start_time.elapsed());
```

In any normal network condition, quorum is reached **before** the outer timeout fires `elapsed < timeout`. With the inverted formula, `elapsed.saturating_sub(timeout)` underflows and `saturating_sub` clamps the result to `Duration::ZERO`. The subsequent guard:

Copy Code

```
if !wait_for_quorum.is_empty() && !remaining_time.is_zero() {
```

evaluates the `!remaining_time.is_zero()` branch as `false`, so the drain task block is entirely skipped. The `quorum-remainder` task is **dead code** under all normal operating conditions.

Code Location

Code of `verify_batch` function from `crates/consensus/worker/src/quorum_waiter.rs` file:

Copy Code

```
171 loop {
172     if let Some(res) = wait_for_quorum.next().await {
173         match res? {
174             Ok(stake) => {
175                 total_stake += stake;
176                 available_stake -= stake;
177                 if total_stake >= threshold {
178                     let remaining_time =
179                         start_time.elapsed().saturating_sub(timeout);
180                     if !wait_for_quorum.is_empty() && !remaining_time.is_zero() {
181                         // Let the remaining waiters have a chance for the remaining
182                         // time.
183                         // These are fire and forget, they will timeout soon so no
184                         // big deal.
185                         spawner_clone.spawn_task("quorum-remainder", async move {
186                             let _ =
187                                 tokio::time::timeout(remaining_time, async move {
188                                     while (wait_for_quorum.next().await).is_some() {
189                                         // do nothing
190                                     }
191                                 })
192                                 .await;
193                         });
194
```

```
                    }
                    break Ok(());
                }
            }
```

Impact

Each invocation of `verify_batch` spawns up to `committee_size − 1` independent per-peer waiter tasks via `spawn_task`. These tasks are **not sub-futures** of the outer `tokio::time::timeout` wrapper; they are independent tokio tasks that survive beyond the outer future's completion. Without the drain task being spawned, each such task continues executing until its RPC oneshot resolves or its own internal network timeout fires, rather than being given a gracefully bounded `remaining_time` window.

Under sustained batch throughput, this translates to `(committee_size − 1) × concurrent_batches` orphaned tasks accumulating at any given time, each holding network connection context and task-manager resources for longer than necessary. This creates unnecessary CPU and memory pressure proportional to the number of slow or non-responsive peers. While the quorum decision and its propagation to the primary are unaffected (the bug is confined to the post-quorum cleanup path), the resource waste degrades system efficiency.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

It is recommended to invert the operands in the remaining-time calculation so that it correctly computes the time budget remaining before the outer deadline:

Copy Code

```
// Replace:
let remaining_time = start_time.elapsed().saturating_sub(timeout);
// With:
let remaining_time = timeout.saturating_sub(start_time.elapsed());
```

This ensures the `quorum-remainder` drain task is spawned with a positive `remaining_time` whenever quorum is reached before the outer timeout fires, allowing the outstanding per-peer waiter futures to complete within a bounded window and releasing their resources promptly.

Remediation Comment

SOLVED: The `ParFin` team solved this issue through the implementation of the recommendation.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/4ecbe3622126e50f3817556f1478e99f5432fc44

7.14 INFINITE RETRY LOOP IN BATCHFETCHER::FETCH ENABLES LIVENESS DENIAL-OF-SERVICE VIA BATCH WITHHOLDING

// LOW

Description

`BatchFetcher::fetch` uses an unbounded `loop` that retries local storage lookups and remote peer fetches indefinitely until every requested batch digest is resolved. There is no cumulative deadline, no maximum retry count, and no cancellation-signal check inside the loop body. Each failed remote attempt incurs a fixed 10-second per-call timeout `safe_request_batches`), after which the loop immediately restarts with no back-off and no termination condition other than full resolution of all digests.

Because the `Subscriber` collects pending `fetch_batches` futures into a `FuturesOrdered` queue, a single stalled future causes **head-of-line blocking**: no subsequent committed SubDag is executed until the hanging fetch resolves. The only cancellation path is a full node shutdown (which broadcasts a `TaskKind::Doomed` signal), making manual operator intervention the sole recovery mechanism.

Code Location

Code of `fetch` function from `crates/consensus/worker/src/batch_fetcher.rs` file:

Copy Code

```
loop {
    if remaining_digests.is_empty() {
        return fetched_batches;
    }

    // Fetch from local storage.
    let _timer = self.metrics.worker_local_fetch_latency.start_timer();
    fetched_batches.extend(self.fetch_local(remaining_digests.clone()).await);
    remaining_digests.retain(|d| !fetched_batches.contains_key(d));
    if remaining_digests.is_empty() {
        return fetched_batches;
    }
    drop(_timer);

    // Fetch from peers.
    let _timer = self.metrics.worker_remote_fetch_latency.start_timer();
    if let Ok(new_batches) =
        self.safe_request_batches(&remaining_digests, Duration::from_secs(10)).await
    {
        // Set received_at timestamp for remote batches.
        let mut updated_new_batches = HashMap::new();
        for (digest, batch) in
            new_batches.iter().filter(|(d, _)| remaining_digests.remove(*d))
        {
            let mut batch = (*batch).clone();
            batch.set_received_at(now());
            updated_new_batches.insert(*digest, batch);
        }
        // Also persist the batches, so they are available after restarts.
        self.batch_store
            .with_write_txn(|txn| {
                for (digest, batch) in &updated_new_batches {
                    txn.insert::<Batches>(digest, batch).map_err(|e| {
                        tracing::error!(target: "batch_fetcher", "failed to insert batch! We c
                        e
                    })?;
                }
                Ok(())
            })
```

```
                .expect("unable to create DB transaction!");
            fetched_batches.extend(updated_new_batches.into_iter());

            if remaining_digests.is_empty() {
                return fetched_batches;
            }
        }
    }
```

Impact

Under **BFT assumptions**, this function is expected to terminate. Any committed batch digest must have been voted for by **≥ 2/3 honest stake**, ensuring that **at least *f + 1* peers** hold the corresponding batch.

However, if a sufficient number of those peers go offline simultaneously **after the SubDag is committed**, the function may **loop indefinitely**. During each iteration, the node performs local lookups and network requests for remote peers, which leads to continuous CPU and networking activity while making no forward progress. As a consequence, the execution pipeline becomes completely stalled until the node is manually restarted, producing the following effects:

- **Liveness loss**: All pending and future consensus outputs are blocked from execution.
- **State divergence risk**: The node falls behind the live network and may require a full re-synchronization after restart.
- **Operational burden**: The only recovery mechanism is an unplanned node restart, as there is no built-in circuit breaker or alerting mechanism for this loop.

It is important to note that **no external actor or single Byzantine validator can reliably trigger this condition without exceeding the BFT safety threshold**.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (2.5)

Recommendation

It is recommended to introduce a **cumulative timeout** and a **maximum retry count** inside `BatchFetcher::fetch`. When the retry budget is exhausted, the function should return either a **partial result** or a **descriptive error**, allowing callers to react appropriately and log the failure instead of blocking indefinitely.

Additionally, a **shutdown or cancellation signal** should be integrated into the loop using `tokio::select!`. This would allow the task to exit cleanly during **node shutdown** or **epoch transitions**, avoiding the need for a full process restart.

Remediation Comment

SOLVED: The **ParFin team** solved this issue through the application of a limit in the number of retries in the batch fetching.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/892a2bada10ba91c0e4c3b14d3d2c48ebc709a6b

7.15 EIP-1559 FEE MARKET PERMANENTLY DISABLED

// LOW

Description

The `adjust_base_fees` function is called at every epoch boundary immediately before the gas accumulator is cleared. Its stated purpose, documented in its own docstring, is to use accumulated gas information to set each worker's base fee for the next epoch. However, the implementation is an explicit no-op.

The function reads `_blocks`, `_gas_used`, and `_base_fee` but assigns nothing. The variables are prefixed with "_" explicitly, and there is no call to `set_base_fee`. The gas accumulator is cleared immediately after.

Code Location

Code of `adjust_base_fees` function from `crates/middleware/orchestrator/src/manager.rs` file:

Copy Code

```
1148 /// Use accumulated gas information to set each workers base fee for the epoch.
1149 /// Currently a no-op.
1150 fn adjust_base_fees(&self, gas_accumulator: &GasAccumulator) {
1151     for worker_id in 0..gas_accumulator.num_workers() {
1152         let worker_id = worker_id as u16;
1153         let (_blocks, _gas_used) = gas_accumulator.get_values(worker_id);
1154         // Change this base fee to update base fee in batches workers create.
1155         let _base_fee = gas_accumulator.base_fee(worker_id);
1156     }
1157 }
```

Code of `await_epoch_execution` function from `crates/middleware/orchestrator/src/manager.rs` file:

Copy Code

```
1164 async fn await_epoch_execution(
1165     &self,
1166     engine: &ExecutionNode,
1167     to_engine: &mpsc::Sender<ConsensusOutput>,
1168     boundary_output: ConsensusOutput,
1169     gas_accumulator: &GasAccumulator,
1170     target_hash: B256,
1171 ) -> eyre::Result<BlockNumHash> {
1172     // subscribe to engine blocks to confirm epoch closed on-chain
1173     let mut executed_output = engine.canonical_block_stream().await;
1174 
1175     // send the boundary output to the engine for execution
1176     to_engine.send(boundary_output).await?;
1177 
1178     let latest = self.consensus_bus.recent_blocks().borrow().latest_block();
1179     // If we have already caught up execution then we are good, skip below loop.
1180     if latest.parent_beacon_block_root == Some(target_hash) {
1181         self.adjust_base_fees(gas_accumulator);
1182         gas_accumulator.clear();
1183         return Ok(latest.num_hash());
1184     }
```

Impact

The absence of base fee adjustment produces several protocol-wide effects:

- **EIP-1559 congestion control is disabled.** Since `base_fee` remains fixed at `MIN_PROTOCOL_BASE_FEE`, transaction pricing does not increase under congestion. As a result, the network lacks the intended demand-regulation mechanism of EIP-1559, and block utilisation is limited only by protocol gas caps rather than market dynamics.
- **Validator fee capture is increased in certain scenarios.** For transactions where `max_fee_per_gas = max_priority_fee_per_gas`, the effective tip can approach the full `max_fee`, since the base fee deduction is negligible. Under a correct EIP-1559 implementation, many of these transactions would instead yield zero tip or be rejected when the base fee is high.
- **Protocol burn or governance revenue becomes negligible.** Because the base fee remains extremely low, the `base_fee × gas_used` component collected per block is effectively zero compared to what an EIP-1559 dynamic base fee would generate during periods of demand.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L (2.5)

Recommendation

It is recommended to implement the `adjust_base_fees` function to compute the next epoch's base fee for each worker using the EIP-1559 adjustment formula applied to the accumulated `gas_used` and `gas_limit` values collected during the epoch. The computed value should be applied via `set_base_fee` on the `BaseFeeContainer` for each worker so that the base fee used by the batch builder, the batch validator, and the execution engine are all consistent for the next epoch.

The transaction pool's `pending_base_fee` should be updated in the same operation to ensure pool admission thresholds remain aligned with the execution base fee. The initialisation path in `manager.rs` should derive the starting base fee from the last executed block's `base_fee_per_gas` field rather than hardcoding `MIN_PROTOCOL_BASE_FEE`, so that a restarted node resumes from the correct market-derived fee level.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by implementing `EIP-1559` dynamic base fee adjustment as a hardfork-gated per-block mechanism: `next_block_base_fee` now applies the standard `calc_next_block_base_fee` formula, and `update_base_fee_after_block` propagates the computed base fee to all workers' `BaseFeeContainer` after every executed block across all execution paths.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/154124cb703c69668b0c5f0c46d442611ed27578

7.16 VALIDATOR SLASHING IS NOT IMPLEMENTED IN THE EXECUTION LAYER

// LOW

Description

The `ConsensusRegistry` contract defines and fully implements `applySlashes(Slash[] calldata slashes)`, protected by the `onlySystemCall` modifier so that only the Rust execution layer can invoke it via `SYSTEM_ADDRESS`. The function decrements validator balances and, when a balance reaches zero, permanently ejects the validator from the protocol via `_consensusBurn()`. The interface explicitly documents the intended call ordering: `applySlashes` must be called before `concludeEpoch`.

Despite this, the Rust execution layer's epoch-close path in `finish()` never calls `applySlashes`. The entire codebase contains zero invocations of this function. A grep across all crates confirms its presence only in `system_calls.rs` as an ABI declaration, never as a `transact_system_call` target.

The interface itself acknowledges this gap with a comment that also reveals the design intent for the transition out of pilot mode:

> *"Not yet enabled during pilot, but scaffolding is included here. For the time being, system calls to this fn can provide empty calldata arrays."*

The absence of even an empty-array call means the execution layer does not follow the protocol's own documented ordering invariant (`applySlashes` → `applyIncentives` → `concludeEpoch`). There is no compile-time flag, runtime warning, or feature gate signalling that this is intentionally omitted.

Code Location

`applySlashes` ABI declaration from `crates/execution/evm/src/system_calls.rs` file:

Copy Code

```
/// Slash information for system calls to decrement outstanding validator balances
/// Currently disabled during MNO pilot.
#[derive(Debug)]
struct Slash {
    /// The validator to slash.
    address validatorAddress;
    /// The amount to slash.
    uint256 amount;
}

// ...

/// Apply negative incentives for the epoch. This must be called before `concludeEpoch`.
function applySlashes(Slash[] calldata slashes) external;
```

Impact

The stake deposited by validators functions as a barrier to entry but provides no active enforcement guarantee during protocol operation. Specifically:

1. **No financial deterrent for misbehavior**: Validators can engage in protocol violations (equivocation, double-signing, coordinated downtime) without any risk to their staked capital. The `Penalty` system (P2P reputation) and `ReputationScores` (leader selection weight) provide network-level and consensus-level consequences, but neither reduces on-chain stake.
2. **Documented ordering invariant violated**: The `IConsensusRegistry` interface requires `applySlashes` to be called before `concludeEpoch`.
3. **Silent gap with no transition path**: The full implementation (slash evidence collection, deterministic propagation across nodes, calldata encoding, and system call invocation) must be built from scratch with no runtime signal that the gap exists.
4. `_consensusBurn()` **is unreachable**: The contract's permanent ejection path for validators who should be removed for gross misbehavior is entirely blocked from the protocol side.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

It is recommended to immediately add a call to `applySlashes(Slash[])` in `finish()` before `applyIncentives` and `concludeEpoch`, initially passing an empty array to comply with the protocol's documented call ordering while preserving the pilot-mode behavior. It is further recommended that before the pilot phase ends, a complete slash evidence mechanism be designed and implemented.

Remediation Comment

RISK ACCEPTED: The **ParFin team** has accepted the risk associated with this finding.

7.17 INTEGER TRUNCATION IN EPOCH COMMITTEE VALIDATION WEAKENS THE MAJORITY THRESHOLD

// INFORMATIONAL

Description

The function `epoch_committee_valid` is called during state-sync epoch record collection to validate that the committee described in a peer-supplied `EpochRecord` is compatible in size and membership with the previously verified committee `prev.next_committee`. The `Ordering::Greater` branch, triggered when the incoming epoch record has fewer members than the known previous committee, applies the following size gate:

```
if epoch_len < 4 || epoch_len < ((committee_len / 3) * 2) {
    return false;
}
```

Rust integer division truncates toward zero before the multiplication is applied. This produces a minimum threshold that is strictly lower than the intended 2/3 majority for any `committee_len` not divisible by three:

`committee_len`	`(committee_len / 3) * 2`	True `ceil(2N/3)`	Gap
7	4	5	-1
10	6	7	-1
13	8	9	-1

An epoch record whose committee size equals `floor(2N/3)` passes the check when the correct threshold is `ceil(2N/3)`, accepting one fewer validator than the mathematical 2/3 majority requires.

Code Location

Code of `epoch_committee_valid` function from `crates/consensus/state-sync/src/epoch.rs` file:

```
13 fn epoch_committee_valid(epoch_rec: &EpochRecord, committee: &[BlsPublicKey]) -> bool {
14     let epoch_len = epoch_rec.committee.len();
15     let committee_len = committee.len();
16     match committee_len.cmp(&epoch_len) {
17         std::cmp::Ordering::Less => false,
18         std::cmp::Ordering::Equal => committee == epoch_rec.committee,
19         std::cmp::Ordering::Greater => {
20             if epoch_len < 4 || epoch_len < ((committee_len / 3) * 2) {
21                 // Make sure we have a reasonable committe size, i.e. don't let
22                 // a bogus record with one signer through, etc.
23                 false
24             } else {
25                 for k in &epoch_rec.committee {
26                     if !committee.contains(k) {
27                         return false;
28                     }
29                 }
30                 true
```

```
            }
        }
    }
}
```

Impact

Independent exploitation of the truncation flaw is blocked by the `verify_with_cert` BLS signature check that follows `epoch_committee_valid` in the same condition, so the impact is confined to a weakening of a defence-in-depth size guard: the check is intended as a sanity gate to reject obviously undersized committees before cryptographic verification, and the truncation silently reduces that gate by one validator below the intended mathematical bound.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

It is recommended to replace the truncating threshold expression with a comparison that avoids integer division entirely:

Copy Code

```
// Before (truncates):
if epoch_len < 4 || epoch_len < ((committee_len / 3) * 2) {

// After (exact 2/3 ceiling, no division, no overflow for realistic committee sizes):
if epoch_len < 4 || epoch_len * 3 < committee_len * 2 {
```

For additional clarity, the equivalent using `div_ceil` (stable since Rust 1.73) is also acceptable:

Copy Code

```
let required = (committee_len * 2).div_ceil(3);
if epoch_len < 4 || epoch_len < required {
```

Remediation Comment

SOLVED: The **ParFin team** solved this issue by the implementation of the recommended remediation.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/238

7.18 REDUNDANT GOSSIP TOPIC SUBSCRIPTION

// INFORMATIONAL

Description

During worker network initialization, the worker batch gossip topic is subscribed twice in immediate succession. The first call uses `subscribe` with no publisher list, and the second uses `subscribe_with_publishers` with the committee key set. Because both calls go through the same network command channel and are processed sequentially by the swarm event loop, the `authorized_publishers` map for the `batch` topic transitions through an intermediate state (None, accept from anyone) before being overwritten with the intended Some(committee_keys) restriction.

Code Location

Snipppet of code of the `spawn_worker_network_for_epoch` from `crates/middleware/orchestrator/src/manager.rs` file:

Copy Code

```
1757 // refresh gossipsub mesh for worker topics to ensure peers are grafted
1758 // after restart/recovery (unsubscribe + subscribe forces gossipsub to
1759 // re-announce the subscription to connected peers)
1760 let txn_topic = LibP2pConfig::worker_txn_topic();
1761 info!(target: "epoch-manager::gossipsub", ?txn_topic, "subscribing to worker txn topic");
1762 network_handle.inner_handle().subscribe(txn_topic.clone()).await?;
1763
1764 // Get gossip from committee members about batches.
1765 // Useful for non-CVVs to prefetch and harmless for CVVs.
1766 let batch_topic = LibP2pConfig::worker_batch_topic();
1767 let _ = network_handle.inner_handle().subscribe(batch_topic.clone()).await;  // [!] redundant:
1768 let committee_keys = committee_keys.into_iter().collect::<HashSet<_>>();
1769 info!(target: "epoch-manager", batch_topic=?batch_topic, committee=?committee_keys, "subscribi
1770 network_handle
1771     .inner_handle()
1772     .subscribe_with_publishers(batch_topic.clone(), committee_keys)           // [!] overwrite
1773     .await?;
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to remove the redundant `subscribe(batch_topic)` call at line 1767. The `subscribe_with_publishers` call is self-contained, which performs the gossipsub-level subscription and sets the authorized publisher entry in one atomic operation, with no intermediate unfiltered state.

Remediation Comment

SOLVED: The **ParFin team** solved this issue through removal of redundant code.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/dbbef416322bd08b65e6793d0b553be8b92f0119

7.19 WORKER GOSSIP HANDLER REDUNDANTLY PROCESSES INVALID MESSAGES TWICE

// INFORMATIONAL

Description

The `process_gossip` function in the worker network handler calls `request_handler.process_gossip()` twice in immediate succession before emitting a reputation penalty against the relay peer.

The second call is nested inside the error branch of the first: if the first attempt returns an `Err`, the function immediately retries with the same message object, the same arguments, no interleaved delay, no state mutation, and no corrective action between the two calls.

Code Location

Code of `process_gossip` function from crates/consensus/worker/src/network/mod.rs file:

Copy Code

```
417 fn process_gossip(&self, msg: GossipMessage, propagation_source: BlsPublicKey) {
418     // clone for spawned tasks
419     let request_handler = self.request_handler.clone();
420     let network_handle = self.network_handle.clone();
421     let task_name = format!("process-gossip-{propagation_source}");
422     self.network_handle.get_task_spawner().spawn_task(task_name, async move {
423         if let Err(e) = request_handler.process_gossip(&msg).await {
424             warn!(target: "worker::network", ?e, "process_gossip");
425             if let Err(e) = request_handler.process_gossip(&msg).await {
426                 warn!(target: "worker::network", ?e, "process_gossip");
427                 // convert error into penalty to lower peer score
428                 if let Some(penalty) = e.into() {
429                     network_handle.report_penalty(propagation_source, penalty).await;
430                 }
431             }
432         }
433     });
434 }
```

Impact

The finding is informational because the practical impact of a 2× deserialization cost is limited and the delay in penalty emission is measured in microseconds.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to remove the inner redundant invocation of `request_handler.process_gossip()` and align the worker handler with the pattern established by the primary handler: attempt processing once, convert the error to a penalty if applicable, and report it immediately.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by removing the redundant code.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/275

7.20 MAINNET GENESIS MISSING HARDFORK TIMESTAMPS

// INFORMATIONAL

Description

The mainnet genesis configuration defines only `shanghaiTime: 0` and omits both `cancunTime` and `pragueTime`. However, the testnet genesis correctly includes all three. Because of this omission, two hardfork guard checks that gate critical pre-block system calls always return `false` on mainnet, causing those calls to be silently skipped on every block from genesis.

The EVM opcode set is also consistently degraded to `SpecId::SHANGHAI`, since `revm_spec_by_timestamp_and_block_number()` evaluates hardfork activation against the same chain spec and falls through to Shanghai when `cancunTime` is absent.

This is a genesis-level misconfiguration. The state is fixed from block 0, immediately observable, and not a latent exploit that can be triggered at a later point. Its severity depends entirely on whether on-chain components read `BEACON_ROOTS_ADDRESS` or rely on Cancun/Prague opcodes.

Code Location

Mainnet genesis configuration in `chain-configs/mainnet/genesis.yaml`:

Copy Code

```
---
config:
  chainId: 487
  homesteadBlock: 0
  daoForkSupport: false
  eip150Block: 0
  eip155Block: 0
  eip158Block: 0
  byzantiumBlock: 0
  constantinopleBlock: 0
  petersburgBlock: 0
  istanbulBlock: 0
  berlinBlock: 0
  londonBlock: 0
  shanghaiTime: 0
  terminalTotalDifficulty: "0x0"
  terminalTotalDifficultyPassed: true
```

Code of `apply_consensus_root_contract_call` function from `crates/execution/evm/src/evm/block.rs` file:

Copy Code

```
424 /// Applies the pre-block call to the EIP-4788 consensus root contract (cancun).
425 fn apply_consensus_root_contract_call(&mut self) -> Result<(), BlockExecutionError> {
426     if !self.spec.is_cancun_active_at_timestamp(self.evm.block().timestamp().saturating_to())
427         return Ok(());   // ← always early-returns on mainnet: BEACON_ROOTS never written
428     }
429     // ...
430     self.evm.transact_system_call(SYSTEM_ADDRESS, BEACON_ROOTS_ADDRESS, parent_beacon_block_ro
431     self.evm.db_mut().commit(res.state);
432     Ok(())
433 }
```

Impact

The absence of `cancunTime` and `pragueTime` in the mainnet genesis has the following functional consequences, all present from block 0:

1. BEACON_ROOTS ring buffer never written
2. EVM opcode set limited to Shanghai
3. Extended blockhash history unavailable

Since this misconfiguration is fixed at genesis, all effects are immediately detectable and not exploitable as a latent attack vector.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to add `cancunTime: 0` and `pragueTime: 0` to `chain-configs/mainnet/genesis.yaml`, matching the testnet configuration and the documented Pectra fork intent.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by adding the `cancunTime` and `pragueTime` to the config file.

Remediation Hash

https://github.com/raylsnetwork/axyl/commit/e3ed35e4d9982a1814cb9b51abd7bc9bb84bf06d

7.21 BATCH DIGEST GOSSIPED BEFORE PERMANENT STORAGE COMMIT

// INFORMATIONAL

Description

In `Worker::seal()` function, once quorum has been confirmed by `batch_attest_handle.await`, the batch digest is immediately broadcast to the gossip network via `publish_batch()`. The permanent write of the batch data to the `Batches` store happens only afterwards, following a CEI (Check–Effect–Interact) inversion: the interaction (network gossip) fires before the final effect (durable DB write).

The only scenario with real impact is a node crash occurring strictly between line 310 and line 339: the digest has been announced to the network but the batch data is never durably committed. After restart, `NodeBatchesCache` is cleared at epoch boundaries, leaving the node unable to serve a digest that peers believe exists. The `report_own_batch` call to the primary (line 346) also never executes, so the batch is not registered with the primary either. The probability of a crash landing in this specific window is considered negligible in practice.

Code Location

Code of `seal` function from `crates/consensus/worker/src/worker.rs` file:

Copy Code

```
297 // Wait for our batch to reach quorum or fail to do so.
298 match batch_attest_handle.await {
299     Ok(res) => {
300         match res {
301             Ok(()) => {
302                 // batch reached quorum!
303                 if let Some(tracker) = &self.batch_tracker {
304                     tracker.batch_quorum_reached(digest);
305                 }
306                 // Publish the digest for any nodes listening to this gossip (non-committee
307                 // members). Note, ignore error- this should not
308                 // happen and should not cause an issue (except the
309                 // underlying p2p network may be in trouble but that will manifest quickly).
310                 let _ = self.network_handle.publish_batch(digest).await;
311             }
312             Err(e) => {
313                 return Err(match e {
314                     crate::quorum_waiter::QuorumWaiterError::QuorumRejected => {
315                         BlockSealError::QuorumRejected
316                     }
317                     crate::quorum_waiter::QuorumWaiterError::AntiQuorum => {
318                         BlockSealError::AntiQuorum
319                     }
320                     crate::quorum_waiter::QuorumWaiterError::Timeout => {
321                         BlockSealError::Timeout
322                     }
323                     crate::quorum_waiter::QuorumWaiterError::Network
324                     | crate::quorum_waiter::QuorumWaiterError::DroppedReceiver
325                     | crate::quorum_waiter::QuorumWaiterError::Rpc(_) => {
326                         BlockSealError::FailedQuorum
327                     }
328                 });
329             }
330         }
331     }
332     Err(e) => {
333         error!(target: "worker::batch_provider", "Join error attempting batch quorum! {e}");
334
```

```
335             return Err(BlockSealError::FailedQuorum);
336         }
337     }
338
339     // Now save it to permenant storage
340     if let Err(e) = self.store.insert::<Batches>(&digest, &batch) {
341         error!(target: "worker::batch_provider", "Store failed with error: {:?}", e);
342         return Err(BlockSealError::FatalDBFailure);
    }
```

Impact

The practical security and liveness impact is minimal. This is a robustness and ordering concern rather than a security vulnerability.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to reorder the operations inside the quorum success branch so that `store.insert::<Batches>()` is called before `publish_batch()`. Since quorum has already been fully confirmed by `batch_attest_handle.await` at this point, this reordering introduces no protocol dependency and eliminates the window in which a digest can be publicly announced without its backing data being durably committed.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by saving the batches in permanent storage before publishing them.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/270

7.22 UNCHECKED INTEGER ARITHMETIC ACROSS MULTIPLE MODULES

// INFORMATIONAL

Description

Multiple locations in the codebase perform arithmetic operations on unsigned integer counters and stake accumulators using bare Rust operators (`-=`, `+`, `-`) without `checked_sub`, `saturating_sub`, or `checked_add` guards. The release profile in `Cargo.toml` does not set `overflow-checks = true`, meaning Rust compiles release binaries with wrapping two's-complement behavior on integer underflow. A counter decremented below zero silently becomes `usize::MAX` with no panic, no log entry, and no observable signal at the call site. Four concrete sites are identified:

Instance 1 — `KadStore::remove` **(**`kad.rs:297`**)**
`self.num_records -= 1` executes after a bare `db.remove().is_ok()` check. MDBX's `remove()` returns `Ok(())` as a no-op for non-existent keys, so `is_ok()` evaluates to `true` regardless of whether a real deletion occurred. If the counter reaches zero and a spurious remove is triggered, `num_records` wraps to `usize::MAX`. The `put()` guard (`self.num_records >= self.config.max_records`) then evaluates permanently to `true`, freezing the Kademlia store. All subsequent `process_kad_put_request` calls propagate `Err(MaxRecords)` via the `?` operator, crashing the network task.

Instance 2 — `KadStore::remove_provider` **(**`kad.rs:394`**)**
Structurally identical to instance 1 applied to the provider counter. `self.num_providers -= 1` follows the same `db.remove().is_ok()` pattern and is subject to the same wrapping underflow under equivalent preconditions.

Instance 3 — `QuorumWaiter::verify_batch` **stake subtraction (**`quorum_waiter.rs:176, 200, 203`**)**
`available_stake` is a `u64` (`VotingPower = u64`) that accumulates the sum of all non-self peer stakes at the start of batch verification and is decremented for each peer response. The three decrement sites (`available_stake -= stake`) carry no saturating guard. In normal operation each peer responds exactly once, preventing underflow. However, no runtime enforcement of this invariant exists, and a future change to the response handling loop would silently produce wrapping values that corrupt subsequent quorum calculations.

Instance 4 — `QuorumWaiter::verify_batch` `max_rejected_stake` **computation (**`quorum_waiter.rs:157`**)**
The expression `(available_stake + total_stake) - threshold` chains two unguarded operations. The subtraction is safe only because the quorum threshold is mathematically guaranteed to be less than or equal to total committee voting power, an invariant the compiler cannot enforce and which holds only for correctly formed committees.

Impact

None of the identified arithmetic sites present a realistic path to triggering overflow or underflow under the current code logic.
This finding is therefore a **defensive coding recommendation** rather than an exploitable vulnerability. Its value lies in resilience against future regressions: as the codebase evolves, an arithmetic guard costs nothing at runtime (saturating operations compile to a single CPU instruction) but prevents a class of subtle, silent failures that would be extremely difficult to diagnose in release builds where overflow wraps without any panic or log signal. Enabling `overflow-checks = true` in the release profile would additionally provide a global, zero-annotation safety net consistent with debug build behavior.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to replace all bare arithmetic operations on unsigned counters with their saturating equivalents. Additionally, it is recommended to add `overflow-checks = true` to the `[profile.release]` section in `Cargo.toml`.

Remediation Comment

SOLVED: The **ParFin team** solved this finding by replacing bare arithmetic operators with `saturating_add` and `saturating_sub`. Additionally, `overflow-checks = true` has been enabled in the release profile for workspace crates, with a scoped override that preserves the default wrapping behavior for non-workspace dependencies.

Remediation Hash

https://github.com/raylsnetwork/axyl/pull/310
